

02. The C++ everyone asks about: memory, objects, and the STL

This chapter is about a small set of C++ ideas that show up in every piece of robot code: where a variable lives, who is allowed to change it, who owns it and who throws it away, and the handful of standard containers you reach for every day. On a robot these are not book knowledge. A variable that dies one line too early is a crash in the middle of a corridor, and a loop that never ends is a motor that never stops. Part A teaches each idea with a drawing first. Part B is eight array, interval and hashing problems where you use those ideas for real.

Rough time: 4 days at about 2 hours a day. Day 1 is Part A only; do not start the problems until you can draw the memory pictures in m1, m4 and m5 from memory.

What you need before this chapter (all in chapter 01): variables and types, a function that takes `int&`, `std::vector`, `std::string`, `std::unordered_map`, `std::pair`, `auto [a, b] = ...`, and how to compile with `g++`. If any of those look foreign, go back to chapter 01 first.

Compile anything in this chapter with:

```
g++ -std=c++17 -Wall -Wextra -O2 file.cpp -o /tmp/x && /tmp/x
```

Part A — Memory, objects, and the STL, from zero

m1. Pointers vs references; what `++a` inside `void f(int a)` does

The picture. The computer's memory is a long row of numbered boxes. The number on a box is its *address*. A variable is a box with a name tag on it.

address:	100	101	102	103	104
	5				
name:	a				

`int a = 5;` made the box at address 100, wrote 5 in it, and stuck the name tag `a` on it. (Real addresses are big numbers like 140732920127452, and a real `int` takes four boxes. Small numbers and one box per value are fine for thinking.)

A pointer is a box that holds another box's number. `int* p = &a;` makes a new box, and writes *the address of a* into it.

address:	100	101	102	
	5		100	
name:	a		p	

`p` holds the number 100, which is where `a` lives

`*p = 6;` means "go to the box whose number is in `p`, and write 6 there". Since `p` holds 100, that is box `a`:

address:	100	101	102	
	6		100	
name:	a		p	

`a` changed, `p` did not

A reference is a second name tag stuck on the same box. `int& r = a;` makes no new box at all:

```
address:    100
           +-----+
           |   6   |      one box, two name tags: a and r
           +-----+
name:      a, r
```

After that, `r = 7` and `a = 7` do exactly the same thing, because they name the same box.

The symbols, one by one.

- `&a` when `a` is a variable: "the address of `a`", the number written on `a`'s box.
- `int*` in a type: "a pointer to an `int`", a box that holds an address.
- `*p` when `p` is a pointer: "go to that address". Reading `*p` reads the box `p` points at; writing `*p = 6` writes into it.
- `int&` in a type: "a reference to an `int`", a second name for an existing box. You use `r` exactly like `a`, no star needed.
- `p->x` when `p` points at an object with a field `x`: short for `(*p).x`, "go to the object and take its `x`".
- `nullptr`: the special address that means "points at nothing". A pointer can hold it; a reference cannot. Before you use `*p` on a pointer that might be `nullptr`, check `if (p)`.

All of that in code:

```
int a = 5;
int* p = &a;    // p holds a's address
*p = 6;        // a is now 6
int& r = a;     // r is another name for a
r = 7;         // a is now 7, and so is *p
int* nothing = nullptr;
if (nothing) { *nothing = 1; } // skipped: nothing points at nothing
```

The ++a question. This is the one everybody gets asked and half of people get wrong:

```
void f(int a) { ++a; }

int x = 1;
f(x);
// what is x now?
```

The answer is `1`. Draw it. When `f(x)` is called, `int a` (no `&`, no `*`) means "make a *new* box called `a` and copy the value of `x` into it":

caller's boxes			f's own boxes (made when f starts, thrown away when f returns)	
x:	+-----+ 1 +-----+	copy 1 -->	a:	+-----+ 1 +-----+
				++a makes this box 2
				...then f returns and this box is thrown away
x:	1 unchanged			

`++a` adds one to *f*'s private copy. Nothing ever touches `x`'s box. Then `f` returns and the copy is gone.

If you want the caller to see the change, say so in the signature:

```
void g(int& a) { ++a; }    // a is a second name tag on the caller's box
void h(int* a) { if (a) ++*a; } // a holds the caller's box number; ++*a goes there and adds one

int x = 1;
f(x);    // x == 1    copy
g(x);    // x == 2    reference: g's a IS x
```

```
h(&x);    // x == 3    pointer: we hand over x's address with &x
h(nullptr); // safe, h checks before using
```

The Python you know. In Python `def f(a): a += 1` on an int does not change the caller either, but `def f(lst): lst.append(1)` does. In Python the difference depends on what *kind* of object is passed. In C++ the difference is written in the function's signature: `int` copies, `int&` aliases, `int*` passes an address. You can read the signature and know, without looking at the type of the thing.

Pointer or reference? Three rules that separate them:

1. A reference must be given a box the moment it is born (`int& r;` on its own is an error). A pointer can be born empty (`int* p = nullptr;`).
2. A reference can never be moved to a different box. `r = y;` does *not* re-stick the name tag on y's box; it copies y's value into the box r already names. A pointer can be re-aimed any time: `p = &y;`.
3. A reference can never be "nothing", so you never have to check it. A pointer can be `nullptr`, so you must.

So: use a reference when the thing must exist (most function parameters). Use a pointer when "there is nothing here" is a real possibility, or when you need to re-aim it later.

Try it. In `00_memory_demo.cpp`, change the line `byRef(x);` to `byValue(x);` and predict which `assert` fails before you compile.

In your own words: "A pointer is a box holding another box's address; `&` gets an address, `*` follows one. A reference is a second name for the same box, so it can never be empty or moved. `++a` inside `void f(int a)` changes a private copy; the caller sees nothing unless the parameter is `int&` or `int*`."

m2. const and const-correctness (const&, const methods)

The picture. `const` is a sticker that says "look but do not touch". You can stick it on a box, on a name tag, on a pointer, or on a button of a class.

```
limit:    +-----+
          | 10  | [const]    reading limit is fine; limit = 11 is a compile error
          +-----+
```

`const int limit = 10;` makes a box you can read but never write. The compiler refuses the write *before the program runs*, which is far better than finding out on the robot.

const T& parameters: a name tag with a sticker. Take a function that counts letters in a string. Two ways to write it:

```
std::size_t countA(std::string s)           // copies the WHOLE string every call
std::size_t countA(const std::string& s)    // a second name tag on the caller's string, read-only
```

Draw the second one:

```
caller's box                inside countA
text:    +-----+
          | "banana" | <-- s is a name tag on this same box, with a [const] sticker:
          +-----+                countA can read every letter, but s[0] = 'x' will not compile
```

No copy is made, so it is fast even for a string of a million characters, and the sticker tells whoever reads the signature "this function will not change your string". That is why almost every function in this tutorial takes big things (`std::string`, `std::vector`, matrices) as `const T&`. For tiny things like `int` and `double`, just copy them; the copy is one box.

```
std::size_t countA(const std::string& s) {
    std::size_t n = 0;
```

```

    for (char c : s) if (c == 'a') ++n;
    return n;
}
// countA("banana") == 3

```

const methods: a sticker on a button. A class (m3 explains classes properly) is a box with compartments and buttons. Writing `const` after a method's parameter list promises "pressing this button does not change any compartment":

```

class Pose2D {
public:
    Pose2D(double x, double y) : x_(x), y_(y) {}
    double x() const { return x_; }           // const button: only reads x_
    void translate(double dx) { x_ += dx; }    // ordinary button: changes x_
private:
    double x_, y_;
};

const Pose2D home(0.0, 0.0); // a Pose2D with a [const] sticker on the whole box
double hx = home.x();        // fine: x() promised not to change anything
// home.translate(1.0);      // compile error: translate() made no such promise

```

The rule that follows: on a `const` object, or through a `const&`, you can *only* press `const` buttons. So every method that just reads (`x()`, `size()`, `empty()`, `getMin()`) must be marked `const`, or nobody can call it on a `const Pose2D&`. When that error hits you, the fix is to add `const` to the method, never to remove `const` from the parameter.

const before the star vs after the star. With pointers there are two things you could protect: the box `p` points at, or `p` itself.

<pre>const int* p = &a;</pre>		"pointer to a const int" <code>p</code> may be re-aimed, but you cannot write through it
<pre> p +-----+ 100 -----> +-----+ </pre>	<pre> a +-----+ 5 +-----+ </pre>	[const via p] <code>*p = 7</code> is an error; <code>p = &b</code> is fine
<pre>int* const p = &a;</pre>		"const pointer to an int" you may write through <code>p</code>, but <code>p</code> is glued to box <code>a</code>
<pre> p +-----+ 100 ==glued==> +-----+ </pre>	<pre> a +-----+ 5 +-----+ </pre>	<code>*p = 7</code> is fine; <code>p = &b</code> is an error

The trick to read any of these: read the declaration from right to left. `const int* p` reads "p is a pointer to an int that is const". `int* const p` reads "p is a const pointer to an int".

```

int a = 5, b = 9;
const int* ptrToConst = &a; // cannot do *ptrToConst = 7; can do ptrToConst = &b
int* const constPtr = &a;  // can do *constPtr = 7; cannot do constPtr = &b
*constPtr = 7;             // a is now 7
ptrToConst = &b;           // now looks at b

```

The Python you know. Python has no `const`. You rely on convention: a function "should not" change the list you pass it. C++ makes the promise part of the type, and the compiler checks it.

(You can skip this on a first read.) Sometimes a `const` method needs to update one compartment that is not really part of the object's state, such as a cache hit counter or a mutex. Mark that one member `mutable` and the compiler allows it inside `const` methods. `00_memory_demo.cpp` has a `hits()` method that does exactly this.

Try it. In `00_memory_demo.cpp`, uncomment the line `cp.translate(1.0);` and read the compiler's error message. It will tell you exactly which promise was broken.

In your own words: "`const` is a look-but-do-not-touch sticker the compiler enforces. I pass big objects as `const T&` so nothing is copied and the signature says I will not change them. Methods that only read are marked `const` so they can be called on `const` objects. `const int*` protects the target, `int* const` protects the pointer; read it right to left."

m3. Classes: constructor, destructor, RAI

The picture. A class is a box with named compartments (the data members) and buttons on the outside (the methods). `Pose2D` from m2 has two compartments, `x_` and `y_`, and buttons `x()` and `translate()`.

+-----+		
	Pose2D	
	x_ : [1.0]	
	y_ : [2.0]	
	[x()] [translate(dx)]	
+-----+		

compartments (private: only the buttons reach in)

buttons (public: anyone can press)

`private:` means only the buttons can open the compartments. `public:` means anyone can press the buttons. A `struct` is the same thing with everything public by default; people use `struct` for plain bundles of data and `class` when the compartments need protecting.

Constructor: what happens when the box is made. A constructor is a special method with the class's own name and no return type. It runs once, at the moment the box is created:

```
Pose2D(double x, double y) : x_(x), y_(y) {}
```

The part after the colon, `: x_(x), y_(y)`, is the **member init list**. It means "fill compartment `x_` with `x` and compartment `y_` with `y` as the box is being built". Without it the compiler builds the box with empty compartments and you assign later, which is slower and, for compartments that are references or `const`, not even allowed. Always use the init list.

Destructor: what happens when the box is thrown away. A destructor is named `~ClassName()` and runs *automatically* at the moment the object stops existing. For a local variable that moment is the closing `}` of the block it was declared in. You never call a destructor yourself.

The timeline. Watch two objects being born and dying inside one block:

```
#include <string>
#include <vector>

struct Door {
    Door(std::string name, std::vector<std::string>& log) : name_(name), log_(log) {
        log_.push_back("open " + name_);    // constructor: runs when the Door is made
    }
    ~Door() { log_.push_back("close " + name_); } // destructor: runs when it is thrown away
private:
    std::string name_;
    std::vector<std::string>& log_;
};

std::vector<std::string> log;
{
    Door a("A", log);
    Door b("B", log);
    log.push_back("work");
}
// log == {"open A", "open B", "work", "close B", "close A"}
```

```

time --->
{   Door a("A")   Door b("B")   log.push_back("work")   }
    |             |             |
    a is made     b is made     the block ends:
    "open A"      "open B"      b is thrown away -> "close B"
                                then a             -> "close A"

```

Two things to notice. The destructors ran without anyone calling them. And they ran in *reverse* order: the last thing made is the first thing thrown away, like taking plates off a pile.

RAII: the library book that returns itself. Imagine a library where every book you pick up walks back to its shelf by itself the moment you leave the room. You could never forget to return one. That is RAII (the name is "Resource Acquisition Is Initialisation", which nobody finds helpful; think "the destructor cleans up"). The pattern is: take the resource in the constructor, give it back in the destructor. Then the resource is always returned, on every way out of the block, including an early `return` and an exception.

You already use RAII without knowing it:

- `std::vector` takes memory in its constructor and gives it back in its destructor.
- `std::ifstream` opens the file in its constructor and closes it in its destructor.
- `std::lock_guard` locks a mutex in its constructor and unlocks it in its destructor.
- The smart pointers in m4 delete the object they own in their destructor.

The Python you know. `__init__` is the constructor. Python has `__del__` but you cannot rely on *when* it runs, so Python invented with `open(...)` as `f`: to get guaranteed cleanup. In C++, every object is inside an invisible with block: the cleanup happens at the `}`.

Rule of zero, in plain words. If every compartment of your class is something that already cleans up after itself (`std::string`, `std::vector`, `std::unique_ptr`, plain numbers), then do not write a destructor at all. The compiler writes one that throws away each compartment in turn, and it is correct. Most classes you write should have zero of: destructor, copy constructor, copy assignment, move constructor, move assignment. Only a class that holds a raw resource (a file handle from a C library, a raw `new`) needs to write them, and then it needs all five.

(You can skip this on a first read.) If a class has any `virtual` method and might be deleted through a pointer to its base class (a `std::unique_ptr<Sensor>` that really holds a `Lidar`), the base class destructor must be declared `virtual ~Sensor() = default;`. Otherwise only the base part is thrown away and the derived part leaks silently. -Wall -Wextra does not warn about this. `00_memory_demo.cpp` shows the correct form.

Try it. In `00_memory_demo.cpp`, add a third `ScopeLog c("C", log);` after `b` and write down the seven log entries you expect, in order, before running it.

In your own words: "A class is a box with compartments and buttons. The constructor runs when the box is made and fills the compartments through the `init` list; the destructor runs by itself when the box goes out of scope, in reverse order of creation. RAII means the destructor returns whatever the constructor took, so cleanup cannot be forgotten. If all my members clean up after themselves, I write no destructor: rule of zero."

m4. `std::unique_ptr` vs `std::shared_ptr`, when to use each

The picture: who owns the toy. An object on the heap (m5 explains the heap; for now: an object that lives outside any function's boxes) is a toy on the floor. Somebody has to be responsible for putting it in the bin when it is no longer needed. C++ makes you say who.

`std::unique_ptr`: one owner. Exactly one child holds the toy. When that child leaves the room (the owner goes out of scope), the toy goes in the bin, automatically, by the destructor (m3). You cannot photocopy a `unique_ptr`; you can only *hand it over* with `std::move`, after which the old holder's hands are empty (`nullptr`).

auto u = std::make_unique<Toy>();	u ---> [Toy]	u owns it
auto v = std::move(u);	u ---> (nothing)	handed over
	v ---> [Toy]	v owns it now
} v leaves the room	[Toy] goes in the bin	

std::shared_ptr: a counter of owners. The toy has a counter sticker: how many children are holding it right now. Every new holder adds one, every leaver subtracts one, and when the counter hits zero the toy goes in the bin.

auto s1 = std::make_shared<Toy>();	[Toy count 1]	held by: s1	
{			
auto s2 = s1;	[Toy count 2]	held by: s1 s2	
}	[Toy count 1]	held by: s1	s2 left the room
s1.reset();	[Toy count 0]	-> bin	s1 let go

std::weak_ptr: may look, does not count. A child who is allowed to look at the toy but is not holding it, so they do not keep it out of the bin. Before using it they must ask "is it still there?" with `w.lock()`, which gives a `shared_ptr` (count goes up by one while they use it) or `nullptr` if the toy is gone. `w.expired()` is the yes/no version of the same question.

All of it in code:

```
#include <memory>

struct Toy { int id = 0; };

std::unique_ptr<Toy> u = std::make_unique<Toy>(); // build the toy and hand me the only owner
u->id = 7; // -> works through smart pointers too
std::unique_ptr<Toy> v = std::move(u); // v owns it now; u == nullptr
// v is thrown away at the end of its block, and the Toy with it

std::shared_ptr<Toy> s1 = std::make_shared<Toy>(); // count 1
{
    std::shared_ptr<Toy> s2 = s1; // count 2
} // s2 gone: count 1
std::weak_ptr<Toy> w = s1; // looks, does not count
bool alive = !w.expired(); // true
s1.reset(); // count 0: Toy is deleted
alive = !w.expired(); // false: w knows it is gone
```

The symbols.

- `std::make_unique<Toy>()` and `std::make_shared<Toy>()`: build a `Toy` on the heap and give you the owner in one step. The arguments in the brackets go to `Toy`'s constructor. You never write `new` yourself.
- `std::move(u)`: "I am done with u, take what it holds". After it, `u` is empty.
- `u->id`: same arrow as a raw pointer (`m1`).
- `u.get()`: the raw address inside, for lending to a function that only *uses* the toy.
- `s1.use_count()`: read the counter. `s1.reset()`: let go.

Which one? `unique_ptr` is the default. It costs exactly the same as a raw pointer, and the ownership is obvious from reading the code. Reach for `shared_ptr` only when you honestly cannot say at compile time who will be the last one using the object: a point cloud handed to three processing stages that finish in any order, a message fanned out to several subscribers. `weak_ptr` is for a child that points back at its parent, so the two do not keep each other alive forever.

Borrowing is fine. A function that only uses the toy and never bins it should take a plain reference (`const Toy&`) or a raw pointer (`Toy*`). Raw pointers are not evil; raw *owning* pointers are, because nothing bins the toy for you. Lend with `u.get()` or `*u`.

The Python you know. Every Python variable is a `shared_ptr`: the object lives while anything refers to it, and a counter decides when it dies. C++ gives you that with `shared_ptr`, but asks you to prefer the cheaper and clearer single owner when you can.

Robotics. A sensor driver object is owned by exactly one node (`unique_ptr`), and its callbacks get a reference to it. In ROS 2 a node is usually a `std::shared_ptr<rcpp::Node>` because both the executor and your code need to keep it alive, and a subscription receives each message as `std::shared_ptr<const Msg>` so one buffer fans out to many callbacks with no copying.

(You can skip this on a first read.) Two traps. Making two `shared_ptr`s from the *same raw pointer* (`std::shared_ptr<T> a(p); std::shared_ptr<T> b(p);`) creates two separate counters, and the toy is binned twice. And using `u` after `std::move(u)` dereferences a `nullptr`. Also, every copy of a `shared_ptr` is an atomic add on the counter, cheap but not free.

Try it. In `00_memory_demo.cpp`, add a second `std::shared_ptr<Sensor> s3 = s1;` inside the inner block and predict the two `use_count()` values.

In your own words: "`unique_ptr` means one owner; it is handed over with `std::move` and the object is deleted when the owner goes out of scope. `shared_ptr` keeps a count of owners and deletes at zero; `weak_ptr` watches without counting. I create both with `make_unique` and `make_shared`, use `unique_ptr` by default, and lend the object to functions as a reference or raw pointer."

m5. Stack vs heap, new/delete and why you avoid them

The picture. There are two places a box can live.

The stack is a pile of trays. Every time a function is called, a tray is put on top of the pile with that function's local boxes on it. When the function returns, the tray is lifted off and every box on it is gone. No bookkeeping, no cleanup, and very fast: putting on a tray is one subtraction.

main() calls f()	f() calls g()	g() returns	f() returns
+-----+	+-----+	+-----+	+-----+
f: a	g: y	f: a	
main: x	f: a	main: x	main: x
+-----+	+-----+	+-----+	+-----+
	g's tray on top	g's tray taken off	f's tray taken off

Every `int a` or `Pose2D p` you declare inside a function lives on its tray.

The heap is a big warehouse. When you need a box that must outlive the current function, or whose size you only know while the program runs (a vector of however many points the lidar returned), you ask the warehouse for a shelf. The old way was `new`: it hands you a ticket (a pointer) to the shelf. When you are done you must hand the ticket back with `delete`. Forget, and the shelf stays taken forever: a *memory leak*. Hand the same ticket back twice and the warehouse corrupts its records: a *double free*.

```
int* raw = new int(5);    // ask the warehouse for one int, write 5 in it; raw is the ticket
// ... use *raw ...
delete raw;               // hand the ticket back. Forget this and the shelf is lost forever.
raw = nullptr;            // the ticket is void now; make that visible
```

Why returning a reference to a local is wrong. The tray is gone.

```
// int& bad() { int x = 1; return x; }    // never write this
```

during bad()	after bad() returns
+-----+	+-----+
bad: x=1 <-- ref	<-- the reference still points here,


```
| main |
+-----+
```

```
| main |
+-----+
```

but the tray is gone. Reading it gives garbage or a crash.

g++ -Wall warns about this one ("reference to local variable returned"). Returning by *value* is fine and is what you should do:

```
std::vector<int> good() {
    std::vector<int> v = {1, 2, 3};
    return v;           // handed to the caller, no copy in practice, nothing leaks
}
```

Where a vector really lives. This is the picture that makes everything click. A `std::vector<int> v(1000, 7)` declared in a function puts a *small* object on the tray (a pointer, a size, a capacity) and the 1000 ints on a shelf in the warehouse. The vector's destructor (m3) returns the shelf the moment the tray is lifted.

stack tray		heap warehouse
+-----+		+-----+ ... +-----+
v: ptr ----->		7 7 7 7 ... 7 1000 ints
size = 1000		+-----+ ... +-----+
capacity = 1000		returned to the warehouse by ~vector()
+-----+		

```
{
    std::vector<int> big(1000, 7); // 1000 ints on a shelf in the warehouse
    // big[999] == 7
}
```

// shelf handed back here, automatically

Why we let vector and unique_ptr do new and delete for us. Every new needs exactly one delete on every path out of the function: the normal end, each early return, and any exception. Humans get that wrong. `std::vector`, `std::string`, `std::make_unique` and `std::make_shared` do the new in their constructor and the delete in their destructor, and the destructor runs on every path (m3). So in modern C++ you almost never type new or delete. When you see them, it is old code, or a library with its own rules (Qt widgets), or embedded code that allocates everything at start-up and never frees.

The Python you know. Every Python object is on the heap and the garbage collector is the warehouse clerk who eventually notices nobody is using a shelf. C++ has no clerk. Destructors are the clerk, and they work because they run at a known moment.

(You can skip this on a first read.) Asking the warehouse for a shelf takes tens to hundreds of nanoseconds and can take a lock, and the time is not predictable. In a hard real-time control loop you therefore avoid any heap allocation after start-up: reserve() your vectors once, then only push_back within that capacity. Also, memory from new[] must be freed with delete[], not delete; one more reason to leave it to std::vector.

Try it. In 00_memory_demo.cpp, uncomment int& bad() and compile with -Wall. Read the warning.

In your own words: "Locals live on the stack, a pile of trays that are thrown away when the function returns, so a reference to a local is pointing at a tray that no longer exists. Big or long-lived data lives on the heap, which must be given back. I never write new and delete myself because vector and unique_ptr do them in a constructor and destructor that run on every path."

m6. size_t is unsigned: why while(--n >= 0) loops forever

The picture. `v.size()` does not give you an int. It gives a `std::size_t`, a number that *cannot be negative*. Think of a car's odometer with 20 digits and no minus sign. Drive forward and it counts up. Now imagine driving backwards from 0. There is no -1 to show, so the odometer rolls round to all nines... in binary, to the biggest number it can hold:

```
size_t n = 0;
--n;
```

```
...  <-  3  <-  2  <-  1  <-  0  <-  18446744073709551615  <-  18446744073709551614  <-  ...
                ^
        subtract 1 from 0 and you land at the far right end, not at -1
```

That number, 18446744073709551615, is `SIZE_MAX`, the largest `size_t`. It is roughly 18 quintillion.

The loop that never ends. Now read this with the odometer in mind:

```
std::vector<int> v = {1, 2, 3};
std::size_t n = v.size();
// while (--n >= 0) { use v[n]; }  <-- do not run this
```

`n` is 3, then 2, 1, 0 and the body runs for each: fine so far. Then `--n` from 0 rolls round to 18446744073709551615. Is that `>= 0`? Yes. Every `size_t` is `>= 0`; there is no other kind. So the loop never stops, and `v[18446744073709551615]` reads memory that is not yours and the program crashes. `-Wextra` does warn ("comparison of unsigned expression in `>= 0` is always true"), which is one of the reasons this tutorial always compiles with it.

The same odometer bites in `v.size() - 1` when `v` is empty:

```
std::vector<int> empty;
std::size_t wrapped = empty.size() - 1;    // 0 - 1 rolls round to 18446744073709551615
// for (std::size_t i = 0; i < empty.size() - 1; ++i)  would run "forever" (it crashes first)
```

Two safe ways to loop backwards.

```
std::vector<int> v = {1, 2, 3};
int sum = 0;

// Way 1: turn the size into an int once, then use ordinary ints. This is the tutorial's habit.
for (int i = static_cast<int>(v.size()) - 1; i >= 0; --i) sum += v[i];
// sum == 6

// Way 2: test first, then subtract. Read "i-- > 0" as "while i is still above 0, step down".
sum = 0;
for (std::size_t i = v.size(); i-- > 0;) sum += v[i];
// sum == 6
```

Way 2 works because the comparison `i > 0` happens *before* the `--`. When `i` is 1 the test passes, `i` becomes 0, the body sees `v[0]`. When `i` is 0 the test fails and the loop ends before any roll-round can happen.

The rule this whole tutorial follows. At the top of a function write `const int n = static_cast<int>(v.size());` and use `int` for every index after that. `static_cast<int>(...)` is C++ for "convert this to an int, and yes I mean it". Problem sizes on LeetCode fit in an `int` comfortably. And never write `size() - 1` without first checking `empty()`.

The Python you know. Python integers never roll round; `0 - 1` is `-1` and `range(len(v) - 1, -1, -1)` just works. That is why this trap only appears once you write C++.

(You can skip this on a first read.) Comparing an `int` with a `size_t`, as in `for (int i = 0; i < v.size(); ++i)`, compiles but warns under `-Wall -Wextra (-Wsign-compare)`, because the `int` is silently converted to unsigned first: a negative `i` becomes enormous and the comparison is wrong. Casting the size once to `int` avoids the warning and the surprise.

Try it. In `00_memory_demo.cpp`, change `int n = 3` in the last block to `std::size_t n = 3` and compile with `-Wall -Wextra`. Read the warning, then put it back. Do not run it.

In your own words: "`size()` returns an unsigned `size_t`, and subtracting one from zero rolls round to the biggest possible value instead of going negative, so `--n >= 0` is always true. I cast the size to `int` once at the top and loop with ints, or I use `for (size_t i = n; i-- > 0;)`, and I check for empty before writing `size() - 1`."

The memory demo file

Everything in m1 to m6 is in one file, as small asserts you can step through:

```
// 00_memory_demo.cpp – pointers vs references, const, RAII, unique_ptr/shared_ptr,
// stack vs heap, and the size_t trap (shown safely).
#include <cassert>
#include <cstdint>
#include <iostream>
#include <memory>
#include <string>
#include <vector>

// ----- m1: pointers vs references -----
void byValue(int a) { ++a; }           // ++a touches a private copy; caller sees nothing
void byRef(int& a) { ++a; }           // alias of the caller's variable
void byPtr(int* a) { if (a) ++*a; }   // may be null, must be checked; *a is the pointee

// ----- m2: const-correctness -----
class Pose2D {
public:
    Pose2D(double x, double y) : x_(x), y_(y) {}           // member init list
    double x() const { return x_; }                       // const method: callable on a const Pose2D
    void translate(double dx) { x_ += dx; }                // non-const: mutates
    std::size_t hits() const { ++cacheHits_; return cacheHits_; } // mutable: bookkeeping in a const method
private:
    double x_, y_;
    mutable std::size_t cacheHits_ = 0;
};

// const T& – read-only view, no copy. Passing std::string by value would copy the buffer.
std::size_t countA(const std::string& s) {
    std::size_t n = 0;
    for (char c : s) if (c == 'a') ++n;
    return n;
}

// ----- m3: RAII -----
// The destructor runs when the object leaves scope, in every exit path (return, exception).
struct ScopeLog {
    ScopeLog(std::string name, std::vector<std::string>& log) : name_(std::move(name)), log_(log) {
        log_.push_back("open " + name_);
    }
    ~ScopeLog() { log_.push_back("close " + name_); }
    ScopeLog(const ScopeLog&) = delete; // a resource handle should not be copied
    ScopeLog& operator=(const ScopeLog&) = delete;
private:
    std::string name_;
    std::vector<std::string>& log_;
};

// ----- m4: ownership with smart pointers -----
struct Sensor {
    virtual ~Sensor() = default; // virtual: deleting through a base pointer must run the derived dtor
    virtual std::string name() const = 0;
};
struct Lidar : Sensor {
    std::string name() const override { return "lidar"; }
};

// Non-owning observer: a raw pointer is fine when the callee never deletes.
std::string describe(const Sensor* s) { return s ? s->name() : "none"; }

// ----- m5: stack vs heap -----
// Returning a reference to a local is a dangling reference – never do this.
// int& bad() { int x = 1; return x; } // g++ -Wall warns: reference to local variable
std::vector<int> good() { std::vector<int> v = {1, 2, 3}; return v; } // moved/elided, no copy, no leak

int main() {
    // m1
    int x = 1;
    byValue(x); assert(x == 1);
    byRef(x);   assert(x == 2);
    byPtr(&x);  assert(x == 3);
    byPtr(nullptr); // safe because byPtr checks
    int* p = &x; *p = 10; assert(x == 10);
    int& r = x; r = 11; assert(x == 11); // r cannot be reseated: "r = y" would assign into x
    Pose2D pose(1.0, 2.0);
    Pose2D* pp = &pose;
}
```

```

assert(pp->x() == 1.0);           // -> is (*pp).x()

// m2
const Pose2D cp(5.0, 0.0);
assert(cp.x() == 5.0);           // OK: x() is const
// cp.translate(1.0);           // compile error: non-const method on const object
assert(cp.hits() == 1);         // mutable member changed inside a const method
assert(countA("banana") == 3);
const int* ptrToConst = &x;     // cannot write *ptrToConst; can repoint
int* const constPtr = &x;       // can write *constPtr; cannot repoint
*constPtr = 12; assert(*ptrToConst == 12);

// m3
std::vector<std::string> log;
{
    ScopeLog a("A", log);
    ScopeLog b("B", log);
    log.push_back("work");
}
// destroyed in reverse order: B then A
assert((log == std::vector<std::string>{"open A", "open B", "work", "close B", "close A"}));

// m4
std::unique_ptr<Sensor> owner = std::make_unique<Lidar>(); // sole owner, zero overhead
assert(describe(owner.get()) == "lidar");                 // lend a non-owning view
std::unique_ptr<Sensor> other = std::move(owner);          // ownership transferred, not copied
assert(owner == nullptr && other->name() == "lidar");

std::shared_ptr<Sensor> s1 = std::make_shared<Lidar>();
{ std::shared_ptr<Sensor> s2 = s1; assert(s1.use_count() == 2); }
assert(s1.use_count() == 1);                               // s2 released its share
std::weak_ptr<Sensor> w = s1;                               // observes without owning
assert(!w.expired());
s1.reset();
assert(w.expired());                                       // weak_ptr knows the object is gone

// m5
assert(good().size() == 3);
{
    std::vector<int> heapBacked(1000, 7); // the vector's buffer is on the heap, freed by its dtor
    assert(heapBacked[999] == 7);
}
int* raw = new int(5);                                     // legacy style: you now own a delete
assert(*raw == 5);
delete raw;                                                 // forget this (or throw before it) and you leak
raw = nullptr;

// m6: size_t is unsigned
std::vector<int> empty;
std::size_t wrapped = empty.size() - 1; // 0 - 1 wraps to SIZE_MAX, not -1
assert(wrapped == static_cast<std::size_t>(-1));
// Safe backwards loops:
std::vector<int> v = {1, 2, 3};
int sum = 0;
for (int i = static_cast<int>(v.size()) - 1; i >= 0; --i) sum += v[i]; // idiom 1: cast once
assert(sum == 6);
sum = 0;
for (std::size_t i = v.size(); i-- > 0;) sum += v[static_cast<int>(i)]; // idiom 2: test-then-decrement
assert(sum == 6);
// while (--n >= 0) with std::size_t n would never be false; with int n it is fine:
int n = 3, ticks = 0;
while (--n >= 0) ++ticks;
assert(ticks == 3);

std::cout << "OK 00_memory_demo.cpp\n";
return 0;
}

```

What to look at:

- `byValue`, `byRef`, `byPtr` at the top, and the three calls in `main()` that leave `x` at 1, 2, 3. That is `m1`'s drawing as code.
- `Pose2D`: the `const` method `x()`, the non-const `translate()`, and the commented-out line `cp.translate(1.0)` that would not compile. Also the mutable counter from the skip paragraph.
- `ScopeLog` and the `log` vector: the timeline from `m3`. Check the order "close B" then "close A".
- `Sensor` and `Lidar`: `unique_ptr` handed over with `std::move`, `shared_ptr` counting up and down inside a block, `weak_ptr` noticing when the object is gone.

- `good()` returning a vector by value, `heapBacked` living on a heap shelf, and the one `new/delete` pair in the file, there only to show what you are avoiding.
- The `size_t` block at the end: the rolled-round value, both safe backwards loops, and the `while (--n >= 0)` loop that is safe only because `n` is an `int`.

s1. `std::sort` with custom comparator / lambda

What a lambda is. A lambda is a small function without a name, written right where you need it. Instead of

```
bool bigger(int a, int b) { return a > b; }
// ... somewhere far away ...
std::sort(v.begin(), v.end(), bigger);
```

you write the function inline:

```
std::sort(v.begin(), v.end(), [](int a, int b) { return a > b; });
```

The shape is `[captures](parameters) { body }`. The square brackets are the strange part; the rest is an ordinary function with the name and the return type left out (the compiler works out the return type from the `return` line).

The square brackets: what does the lambda get to see? A lambda can use variables from the function around it, but you must say *how*:

- `[]` : the lambda uses nothing from outside. Most comparators are like this.
- `[k]` : the lambda gets its *own copy* of `k`, taken at the moment the lambda is written.
- `[&k]` : the lambda gets a *name tag on the real k* (a reference, `m1`).
- `[=]` : copy everything from outside that the body mentions. `[&]` : reference everything.

Draw the difference:

```
int k = 2;
auto timesK = [k](int a) { return a * k; }; // copies k: timesK has its own box holding 2
auto bumpK = [&k]() { ++k; };               // refers to k: bumpK's k IS main's k
bumpK();                                   // main's k is now 3
int r = timesK(3);                         // 6, not 9: timesK still holds its copy of 2
```

main's boxes +-----+ k 3 <-- bumpK writes here +-----+	timesK's private box +-----+ 2 copied when +-----+ the lambda was written	bumpK (no box of its own; just a name tag on main's k)
---	---	---

`auto` in front of a lambda is required: the lambda's type has no name you could write. When a comparator needs to look at a big vector from outside, capture it by reference (`[&cost]`) so the vector is not copied into the lambda.

What the comparator must answer. `std::sort` calls your lambda with two elements and expects the answer to one question: "*does a go before b?*" True means "a first". So `a < b` sorts ascending, `a > b` sorts descending.

Sort by the second field. This is the case you meet most: a list of pairs, sort by the number.

```
#include <algorithm>
#include <string>
#include <utility>
#include <vector>
```

```
std::vector<std::pair<std::string, int>> people = {{"ann", 31}, {"bob", 25}, {"cy", 40}};
std::sort(people.begin(), people.end(),
    [](const auto& a, const auto& b) { return a.second < b.second; });
// people is now: bob 25, ann 31, cy 40
```

- `people.begin(), people.end()`: sort the whole vector (s4 explains what these are).
- `const auto& a`: "a read-only name tag on one element, whatever type it is". No copy.
- `a.second`: the number in the pair.

Without a comparator, `std::sort(people.begin(), people.end())` sorts pairs by first and, when those tie, by second. That is often exactly right, for example to sort (row, col) points.

The Python you know. `sorted(people, key=lambda p: p[1])`. Python's key gives back "the thing to sort by". C++'s comparator instead answers "is a before b?". Same idea, different question, and the C++ form lets you say "score descending, then name ascending" in one function.

The rule that keeps sort from crashing. The comparator must say "is a *before* b", never "is a before or equal to b". Write `<`, never `<=`. If you write `<=`, then asked "is a before a?" your lambda says yes, which is nonsense, and `std::sort` is allowed to walk off the end of the array. (The proper name for this rule is *strict weak ordering*. Remember it as: strictly before.)

(You can skip this on a first read.) `std::sort` is $O(n \log n)$ and may reorder elements that compare equal. If equal elements must keep their original order, use `std::stable_sort`. To sort by several keys in one line, compare `std::tie(a.score, a.frame) < std::tie(b.score, b.frame)`; see `00_stl_demo.cpp` for a descending-then-ascending version.

Try it. In `00_stl_demo.cpp`, change the descending comparator to `a >= b` and run it. It may work, may crash, may print garbage: that is what "allowed to do anything" looks like. Put it back.

In your own words: "A lambda is a function without a name, written where it is used; the square brackets say which outside variables it copies or refers to. A sort comparator answers 'does a go before b?' with a strict `<`, never `<=`. To sort pairs by the second field I compare `a.second < b.second`."

s2. `std::priority_queue` (min-heap via `greater<>`)

The picture. A priority queue is a pile where the biggest thing is always on top. You throw items onto the pile in any order; whenever you look, the top is the largest, and when you take the top off, the next largest rises to take its place.

push 3, then 1, then 4:

```
top -> [ 4 ]      top() is 4
        [ 3 ]      pop() removes the 4; now 3 is on top
        [ 1 ]
```

(Inside it is a clever tree called a heap, arranged so that push and pop each cost about $\log n$ steps and looking at the top costs one. You never see the tree; you only touch the top.)

```
#include <queue>

std::priority_queue<int> maxHeap;
for (int e : {3, 1, 4}) maxHeap.push(e);
// maxHeap.top() == 4
maxHeap.pop();      // returns nothing; look with top() first if you need the value
// maxHeap.top() == 3
```

Smallest on top: `std::greater<>`. Most robotics uses (Dijkstra, A*, "k closest") want the *smallest* on top. You get that by telling the pile to rank items backwards, using `std::greater<int>` as the third piece of the type:

```
#include <functional>    // std::greater

std::priority_queue<int, std::vector<int>, std::greater<int>> minHeap;
for (int e : {3, 1, 4}) minHeap.push(e);
// minHeap.top() == 1
minHeap.pop();
// minHeap.top() == 3
```

Three things inside the angle brackets, in this order: the element type, the container the pile is built on (always `std::vector<...>`), and the comparison. `std::greater<int>` means "treat a as lower priority than b when $a > b$ ", so big numbers sink and the smallest floats to the top. If you forget the middle `std::vector<int>` it does not compile.

A pile of pairs sorts by the first element. Pairs compare by `first`, and only on a tie by `second`. So a pile of (distance, node) pairs with `std::greater` keeps the *closest node* on top. That is exactly the frontier of Dijkstra's algorithm (chapter 04).

```
using DistNode = std::pair<double, int>;    // (distance, node id)
std::priority_queue<DistNode, std::vector<DistNode>, std::greater<DistNode>> frontier;
frontier.push({2.5, 7});
frontier.push({0.5, 3});
frontier.push({1.0, 9});
// frontier.top() is {0.5, 3}: the smallest distance, and .second gives node 3
```

- `using DistNode = std::pair<double, int>;` gives a long type a short name, like a Python variable that holds a type.
- `{0.5, 3}` builds the pair on the spot.

The Python you know. `heapq` is always a min-heap on a list; to get a max-heap you push `-x`. C++ is the other way round: max by default, `std::greater` for min. Both have no "peek and pop in one call": in C++ read `top()`, then `pop()`.

(You can skip this on a first read.) For a custom ordering (a struct with a score), write a lambda and pass its type with `decltype(cmp)` and the lambda itself to the constructor: `std::priority_queue<T, std::vector<T>, decltype(cmp)> pq(cmp);`. The lambda's meaning is inverted compared with `std::sort`: returning true means "a sits *below* b". Also, there is no way to change an item's priority once it is in the pile; Dijkstra pushes a fresh entry instead and skips stale ones when they surface.

Try it. In `00_stl_demo.cpp`, push `{0.5, 1}` into `frontier` as well and predict which node is on top (hint: equal `first`, so `second` decides).

In your own words: "A priority queue is a pile with the biggest on top; push and pop are $\log n$, top is constant. For smallest-on-top I write `std::priority_queue<T, std::vector<T>, std::greater<T>>`. A pile of pairs ranks by the first element, so (distance, node) gives me the closest node on top."

s3. `std::deque`, `std::queue`, `std::stack`

The picture. You met two of these in chapter 03, A1b: a **queue** is the line at a shop (join at the back, served from the front) and a **stack** is a pile of plates (put on top, take from the top). The third, a **deque** ("double-ended queue", said "deck"), is a line where people may join or leave at *either* end, and where you may also point at the fifth person in line without walking there.

queue:	front [1 2 3] back	push at the back, pop from the front
stack:	[3] <- top	push on top, pop from the top
	[2]	
	[1]	
deque:	front [0 1 2] back	push or pop at both ends, plus <code>w[1]</code>

The table. Every operation you will use:

	<code>std::stack<T></code>	<code>std::queue<T></code>	<code>std::deque<T></code>
add	<code>push(x)</code> on top	<code>push(x)</code> at the back	<code>push_back(x)</code> , <code>push_front(x)</code>
remove	<code>pop()</code> from top	<code>pop()</code> from front	<code>pop_back()</code> , <code>pop_front()</code>
look	<code>top()</code>	<code>front()</code> , <code>back()</code>	<code>front()</code> , <code>back()</code> , <code>w[i]</code>
how many / any	<code>size()</code> , <code>empty()</code>	<code>size()</code> , <code>empty()</code>	<code>size()</code> , <code>empty()</code>
Python	<code>list.append</code> / <code>list.pop()</code>	<code>deque.append</code> / <code>deque.popleft</code>	<code>collections.deque</code>

```
#include <deque>
#include <queue>
#include <stack>

std::queue<int> q;
q.push(1); q.push(2);
// q.front() == 1
q.pop(); // returns nothing: read front() first if you need it
// q.front() == 2, q.size() == 1

std::stack<int> st;
st.push(1); st.push(2);
// st.top() == 2
st.pop();
// st.top() == 1

std::deque<int> w;
w.push_back(1); w.push_back(2); w.push_front(0);
// w.front() == 0, w.back() == 2, w[1] == 1
w.pop_front(); w.pop_back();
// w.size() == 1, w[0] == 1
```

Two habits to build. First, `pop()` gives nothing back on all three (and on `priority_queue`). Look with `front()` or `top()`, then `pop()`. Second, looking at an empty one (`q.front()` when `q.empty()`) is not an error you can catch; it reads garbage or crashes. Check `empty()` first.

Which one when? `queue` for BFS (chapter 03). `stack` for DFS without recursion, matching brackets, and Min Stack (problem 8 below). `deque` when you need both ends, which is the sliding window maximum in chapter 05.

(You can skip this on a first read.) `std::queue` and `std::stack` are thin wrappers around a `std::deque` that hide the operations you should not use; that is why `queue` has no `q[i]` and no way to loop over it. A `deque` is stored as a chain of fixed-size chunks, which is how it manages constant-time work at both ends *and* `w[i]`.

In your own words: "queue is a shop line: push at the back, pop from the front. stack is a pile of plates: push and pop at the top. deque allows both ends and indexing. `pop()` returns nothing, so I read `front()` or `top()` first, and I check `empty()` before I look."

s4. `std::map` vs `unordered_map`, iterators

The picture: a phone book and a cupboard of buckets. Both containers store key -> value pairs. They differ in *how* they keep them.

<code>std::map<int, std::string></code> (a phone book, sorted by key)	<code>std::unordered_map<int, std::string></code> (a cupboard of buckets)
10 -> "a"	bucket 0: (30, "c")
20 -> "b"	bucket 1: (empty)
30 -> "c"	bucket 2: (10, "a") (20, "b")
walk it: 10, 20, 30, always in order	walk it: some order, never rely on it

find 20: open the middle, log n steps
needs: a < on the key

find 20: hash(20) says "bucket 2", one step
needs: a hash and == on the key

Use `unordered_map` for plain counting and looking up (Two Sum, Top K Frequent). Use `map` when you need the keys in order, or need "the first key at or above x": timestamps, sorted events, interval bookkeeping. Python's `dict` is a hash table like `unordered_map`; there is no built-in sorted dict.

What an iterator is. An iterator is a *bookmark* into a container. It marks one element; you can read the element through it, and move the bookmark forward with `++`.

```
elements:  [10:"a"]  [20:"b"]  [30:"c"]  (nothing here)
            ^                ^
          begin()            end()
```

`begin()` is a bookmark on the first element.

`end()` is a bookmark one past the last: a marker that means "you have run out".

Never read through `end()`. You only compare against it.

For a `map`, `it->first` is the key on the bookmarked page and `it->second` is the value (the arrow from `m1`: the bookmark behaves like a pointer to a pair). `auto` is your friend; iterator types have very long names.

```
#include <map>
#include <string>
#include <unordered_map>

std::map<int, std::string> timeline = {{30, "c"}, {10, "a"}, {20, "b"}};

std::string order;
for (const auto& [t, name] : timeline) order += name; // visits 10, 20, 30: order == "abc"

auto it = timeline.lower_bound(15); // bookmark on the first key >= 15, which is 20
// it->first == 20, it->second == "b"
bool found = timeline.lower_bound(99) != timeline.end(); // false: no key >= 99
```

- `for (const auto& [t, name] : timeline)` is the range-for from chapter 01; each element is a pair and `[t, name]` unpacks it, like Python's `for t, name in d.items()`.
- `lower_bound(x)`: the first key that is `>= x`. Only `map` has it; the cupboard is not sorted.
- Comparing with `end()` is how you ask "did you find anything?".

The erase-while-looping trap. Suppose you walk the phone book and tear out the page for key 20.

```
[10]  [20]  [30]
      ^ e           e is the bookmark on 20

timeline.erase(e);           the page is torn out...

[10]  [30]
      ? e                 ...and e is a bookmark on a page that no longer exists.
                          ++e now reads a torn-out page: garbage or a crash.
```

The fix: `erase` hands you back a fresh bookmark on the *next* page. Use it.

```
for (auto e = timeline.begin(); e != timeline.end(); e = timeline.erase(e)) { // note: no ++e up here
    if (e->first == 20) e = timeline.erase(e); // erase gives the bookmark on 30
    else ++e; // only step when we did not erase
}
// timeline now holds 10 and 30
```

`m[key]` inserts. In both containers, `counts["z"]` on a missing key *creates* the entry with a default value (0 for `int`) and returns it. That is perfect for counting and terrible for a read-only check, because the check quietly grows the map. For "is it there?" use `find` or `count`.

```
std::unordered_map<std::string, int> counts;
for (const char* w : {"a", "b", "a"}) ++counts[w];    // "a" -> 2, "b" -> 1
// counts.size() == 2
bool hasZ = counts.find("z") != counts.end();        // false, and nothing was inserted
int zCount = counts.count("z");                      // 0
```

- `++counts[w]`: look up (or create at 0), then add one. This is Python's `counts[w] = counts.get(w, 0) + 1`, or `Counter`.
- `counts.find("z")` gives a bookmark, or `end()` if absent. It never inserts.

(You can skip this on a first read.) Bookmark arithmetic like `it + 2` or `it2 - it1` works only on containers stored as one block: `vector`, `deque`, `array`. On a map use `std::next(it)` and `std::prev(it)`. Bookmarks can also go stale without you erasing anything: `push_back` on a `vector` may move the whole block to a bigger shelf (m5), after which every bookmark into it is dead. Take bookmarks late and use them quickly.

Try it. In `00_stl_demo.cpp`, change `counts.find("z") == counts.end()` to `counts["z"] == 0` and predict what `counts.size()` becomes.

In your own words: "map is a sorted phone book, log n per operation, with `lower_bound` for 'first key at or above x'. `unordered_map` is a cupboard of buckets, constant time on average, no order. An iterator is a bookmark; `end()` is the one-past-the-end marker I compare against but never read. `m[key]` inserts if missing, so I use `find` for read-only lookups, and when erasing in a loop I use the bookmark that `erase` returns."

s5. `std::array`, `std::string_view` basics

`std::array`: a vector whose size is fixed forever. The size is written into the type, `std::array<double, 3>`, and can never change: no `push_back`, no `resize`. In return there is no heap shelf at all (m5): the elements sit right on the stack tray, and the whole thing is as cheap as three plain doubles. Use it when the size is known while you write the program: an xyz point, a 4x4 transform, the four neighbour offsets, a tiny fixed grid.

<code>std::array<double, 3> xyz</code>	stack tray: [1.0 2.0 3.0]	no pointer, no warehouse
<code>std::vector<double> v</code>	stack tray: [ptr size cap]	--> warehouse: [1.0 2.0 3.0]

```
#include <array>

std::array<double, 3> xyz = {1.0, 2.0, 3.0};
// xyz.size() == 3, xyz[2] == 3.0
std::array<std::array<int, 3>, 3> grid{};    // a 3x3 grid, all zeros: the {} does the zeroing
grid[1][1] = 9;
// grid[1][1] == 9, grid[0][0] == 0
```

Everything you know from `vector` still works: `size()`, `[i]`, `range-for`, `begin()/end()`.

`std::string_view`: a window onto someone else's string. A `string_view` does not own any characters. It is two numbers: where the characters start, and how many there are. Making one copies nothing; cutting a piece out of one (`substr`) copies nothing either.

```
std::string topic = "/sensors/lidar/points";    topic owns 21 characters on a heap shelf

std::string_view sv = topic;                    sv = [ start: topic's first char | length: 21 ]
std::string_view part = sv.substr(9, 5);        part = [ start: the 'l' of lidar | length: 5 ] -> "lidar"

/ s e n s o r s / l i d a r / p o i n t s
^           ^-----^
sv starts here    part is this window
```

```
#include <string>
#include <string_view>

bool startsWith(std::string_view s, std::string_view prefix) {
    return s.substr(0, prefix.size()) == prefix;
}

std::string topic = "/sensors/lidar/points";
std::string_view sv = topic;           // a window onto topic; nothing copied
// startsWith(sv, "/sensors") == true
// sv.substr(9, 5) == "lidar"          // another window, still nothing copied
// startsWith("literal", "lit") == true // a quoted literal becomes a string_view for free
```

Taking `std::string_view` as a parameter (instead of `const std::string&`) means callers with a `std::string`, a quoted literal, or a `char*` all work without building a temporary string.

Why the window goes dark. The window shows characters that belong to someone else. If that someone dies, the window is looking at a returned warehouse shelf (m5).

```
// std::string_view dark = std::string("temp"); // the temporary string dies at the ';'
//                                           // dark now looks at freed memory. Never do this.
```

Same trap in disguise: returning a `string_view` onto a local `std::string`, or storing a `string_view` in a struct that outlives the string. The rule: only use a `string_view` while you can point at the `std::string` (or literal) that owns the characters and know it is still alive. Literals live for the whole program, so views of them are always safe.

The Python you know. Python strings are immutable and every slice `s[9:14]` copies. A `string_view` slice is free but borrows. `std::array` is a tuple of fixed length that you can still write into.

Try it. In `00_stl_demo.cpp`, change `sv.substr(9, 5)` to `sv.substr(1, 7)` and predict the string before you run it.

In your own words: "`std::array` is a vector whose size is part of the type and never changes, with no heap allocation; I use it for small fixed-size things like points and direction tables. `string_view` is a window onto characters someone else owns: no copies, free slicing, but it must not outlive the string it looks at."

The STL demo file

Everything in `s1` to `s5`, as asserts:

```
// 00_stl_demo.cpp - sort with lambdas, priority_queue, deque/queue/stack, map vs unordered_map,
// std::array and std::string_view.
#include <algorithm>
#include <array>
#include <cassert>
#include <cstdint>
#include <deque>
#include <functional>
#include <iostream>
#include <map>
#include <numeric>
#include <queue>
#include <stack>
#include <string>
#include <string_view>
#include <unordered_map>
#include <utility>
#include <vector>

struct Detection {
    std::string label;
    double score;
    int frame;
};

// string_view: a (pointer, length) view of characters someone else owns. No copy, no allocation.
bool startsWith(std::string_view s, std::string_view prefix) {
```

```

    return s.substr(0, prefix.size()) == prefix;
}

int main() {
    // ----- s1: std::sort with custom comparator / lambda -----
    std::vector<int> v = {5, 2, 9, 1};
    std::sort(v.begin(), v.end()); // ascending, uses operator<
    assert((v == std::vector<int>{1, 2, 5, 9}));
    std::sort(v.begin(), v.end(), [](int a, int b) { return a > b; }); // descending; comparator = "a goes before b?"
    assert((v == std::vector<int>{9, 5, 2, 1}));

    // pairs sort lexicographically out of the box: by first, then second
    std::vector<std::pair<int, int>> pts = {{2, 1}, {1, 5}, {1, 2}};
    std::sort(pts.begin(), pts.end());
    assert((pts == std::vector<std::pair<int, int>>{{1, 2}, {1, 5}, {2, 1}}));

    // structs: multiple keys – score descending, then frame ascending. std::tie makes this one line.
    std::vector<Detection> dets = {{{"car", 0.9, 3}, {"person", 0.9, 1}, {"bike", 0.5, 2}};
    std::stable_sort(dets.begin(), dets.end(), [](const Detection& a, const Detection& b) {
        return std::tie(b.score, a.frame) < std::tie(a.score, b.frame);
    });
    assert(dets[0].label == "person" && dets[1].label == "car" && dets[2].label == "bike");

    // argsort: sort indices by the values they point at – the lambda captures `cost` by reference
    std::vector<double> cost = {3.5, 0.2, 1.7};
    std::vector<int> idx(cost.size());
    std::iota(idx.begin(), idx.end(), 0);
    std::sort(idx.begin(), idx.end(), [&cost](int a, int b) { return cost[a] < cost[b]; });
    assert((idx == std::vector<int>{1, 2, 0}));

    // captures: [=] copies, [&] references, [k] copies just k
    int k = 2;
    auto timesK = [k](int a) { return a * k; };
    auto bumpK = [&k]() { ++k; };
    bumpK();
    assert(timesK(3) == 6 && k == 3); // timesK captured k when it was 2

    // ----- s2: std::priority_queue -----
    std::priority_queue<int> maxHeap;
    for (int e : {3, 1, 4}) maxHeap.push(e);
    assert(maxHeap.top() == 4);

    std::priority_queue<int, std::vector<int>, std::greater<int>> minHeap; // min-heap via greater<>
    for (int e : {3, 1, 4}) minHeap.push(e);
    assert(minHeap.top() == 1);
    minHeap.pop(); // pop() returns void
    assert(minHeap.top() == 3);

    // min-heap of (distance, node) – Dijkstra's frontier. pair compares by first, so distance wins.
    using DistNode = std::pair<double, int>;
    std::priority_queue<DistNode, std::vector<DistNode>, std::greater<DistNode>> frontier;
    frontier.push({2.5, 7}); frontier.push({0.5, 3}); frontier.push({1.0, 9});
    assert(frontier.top().second == 3);

    // custom comparator: lambda + decltype, pass the lambda to the constructor
    auto byScoreAsc = [](const Detection& a, const Detection& b) { return a.score > b.score; }; // "a is lower priority than b"
    std::priority_queue<Detection, std::vector<Detection>, decltype(byScoreAsc)> lowScoreFirst(byScoreAsc);
    for (const auto& d : dets) lowScoreFirst.push(d);
    assert(lowScoreFirst.top().label == "bike");

    // ----- s3: std::deque, std::queue, std::stack -----
    std::queue<int> q; // adapter over deque: push back, pop front (BFS)
    q.push(1); q.push(2);
    assert(q.front() == 1);
    q.pop(); // void – read front() first if you need the value
    assert(q.front() == 2 && q.size() == 1);

    std::stack<int> st; // adapter over deque: push/pop/top at the back (DFS, parentheses)
    st.push(1); st.push(2);
    assert(st.top() == 2);
    st.pop();
    assert(st.top() == 1);

    std::deque<int> window; // both ends O(1), plus random access – sliding-window max
    window.push_back(1); window.push_back(2); window.push_front(0);
    assert(window.front() == 0 && window.back() == 2 && window[1] == 1);
    window.pop_front(); window.pop_back();
    assert(window.size() == 1 && window[0] == 1);

    // ----- s4: std::map vs unordered_map, iterators -----
    std::map<int, std::string> timeline = {{30, "c"}, {10, "a"}, {20, "b"}}; // red-black tree, key order
    std::string order;
    for (const auto& [t, name] : timeline) order += name; // iterates 10, 20, 30
    assert(order == "abc");
    auto it = timeline.lower_bound(15); // first key >= 15 – O(log n), only on ordered containers
    assert(it != timeline.end() && it->first == 20);
    assert(timeline.lower_bound(99) == timeline.end());
    assert(timeline.rbegin()->first == 30); // reverse iterator: largest key

```

```

// erase-while-iterating: erase() returns the next valid iterator; ++it after erase is a bug
for (auto e = timeline.begin(); e != timeline.end(); ) {
    if (e->first == 20) e = timeline.erase(e);
    else ++e;
}
assert(timeline.size() == 2 && timeline.count(20) == 0);

std::unordered_map<std::string, int> counts; // hash table, no order, O(1) average
for (const char* w : {"a", "b", "a"}) ++counts[w]; // operator[] inserts 0 if missing
assert(counts["a"] == 2 && counts.size() == 2);
assert(counts.find("z") == counts.end()); // find() does not insert; operator[] would
assert(counts.count("z") == 0 && counts.size() == 2);

// iterator arithmetic (it + 2, it - begin) only on random-access containers (vector, deque, array)
std::vector<int> w = {4, 5, 6};
auto mid = w.begin() + 1;
assert(*mid == 5 && mid - w.begin() == 1);
assert(std::next(timeline.begin()) == std::prev(timeline.end())); // for map, use std::next/std::prev

// ----- s5: std::array, std::string_view -----
std::array<double, 3> xyz = {1.0, 2.0, 3.0}; // size is part of the type, lives on the stack
assert(xyz.size() == 3 && xyz[2] == 3.0);
std::array<std::array<int, 3>, 3> grid{}; // fixed 3x3, zero-initialised: a tiny costmap
grid[1][1] = 9;
assert(grid[1][1] == 9 && grid[0][0] == 0);

std::string topic = "/sensors/lidar/points";
std::string_view sv = topic; // views topic's buffer; topic must outlive sv
assert(startsWith(sv, "/sensors"));
assert(sv.substr(9, 5) == "lidar"); // substr on a view is O(1): no allocation
assert(startsWith("literal", "lit")); // literals convert too

std::cout << "OK 00_stl_demo.cpp\n";
return 0;
}

```

What to look at:

- The four `std::sort` calls at the top: no comparator, a descending lambda, pairs sorting themselves, and the `argsort` that captures `cost` by reference with `[&cost]`.
- `timesK` and `bumpK`: the copy-versus-reference capture drawing from `s1`, as an assert.
- `maxHeap`, `minHeap` and `frontier`: the three pile shapes from `s2`. Note `pop()` returns nothing.
- The `queue`, `stack` and `deque` block: the table from `s3`, one line per cell.
- `timeline`: range-for in key order, `lower_bound`, `rbegin()` for the largest key, and the erase-while-looping loop written the safe way.
- `counts`: `++counts[w]` inserting, and `find` not inserting.
- `xyz`, `grid`, `sv`: the array and the string window from `s5`.

Part B — The problems

Each problem follows the same shape: the full problem in plain words with a drawn example, how you would solve it by hand, the Python you would have written, the C++ solution with every new construct explained, a walk-through of the example, and the follow-up questions that usually come with it.

The rule for using this part: read the problem statement, close the file, and try it in C++ for 25 minutes. Then read the rest.

1. Product of Array Except Self (LeetCode 238, medium)

The problem in plain words. You are given a list of integers `nums`. Build a new list `answer` of the same length where `answer[i]` is the product of every number in `nums` except `nums[i]`. You are not allowed to use division, and the whole thing must take time proportional to the length of the list.

index:	0	1	2	3	
nums:	1	2	3	4	
answer:	24	12	8	6	answer[1] = 1 * 3 * 4 = 12 (everything but the 2)

Edge cases: a zero in the list makes every other slot 0, and the zero's own slot gets the product of the rest ([1, 1, 0, -3, 3] gives [0, 0, 9, 0, 0]). Two zeros make everything 0. The list has at least 2 numbers and at most 100,000, and the problem promises every product fits in a 32-bit `int`.

By hand. For each slot, cover it with your finger and multiply the numbers on the left of your finger by the numbers on the right. For slot 1 that is 1 on the left and $3 * 4 = 12$ on the right: 12. Doing that for every slot is slow (each slot re-multiplies almost the whole list), but it shows the shape: *left product times right product*. So compute all the left products in one sweep, all the right products in another sweep, and multiply.

nums:	1	2	3	4	
left of i:	1	1	2	6	nothing to the left of slot 0 counts as 1
right of i:	24	12	4	1	nothing to the right of slot 3 counts as 1
multiply:	24	12	8	6	

The idea. 1. Pass 1, left to right: `out[i] = out[i-1] * nums[i-1]`, with `out[0] = 1`. After this, `out` holds the left products. 2. Pass 2, right to left, carrying a running suffix that starts at 1: multiply `out[i]` by suffix, then fold `nums[i]` into suffix. 3. The output list is reused as the left-product list, so the only extra memory is suffix.

The Python you would have written.

```
def productExceptSelf(nums):
    n = len(nums)
    out = [1] * n
    for i in range(1, n):
        out[i] = out[i - 1] * nums[i - 1]    # product of everything left of i
    suffix = 1
    for i in range(n - 1, -1, -1):
        out[i] *= suffix                    # times product of everything right of i
        suffix *= nums[i]
    return out
```

In C++.

```
// 01_product_except_self.cpp - LeetCode 238. Prefix/suffix products, no division, O(1) extra space.
#include <cassert>
#include <iostream>
#include <vector>

std::vector<int> productExceptSelf(std::vector<int>& nums) {
    const int n = static_cast<int>(nums.size());
    std::vector<int> out(n, 1);
    // Pass 1: out[i] = product of everything to the left of i.
    for (int i = 1; i < n; ++i) out[i] = out[i - 1] * nums[i - 1];
    // Pass 2: walk right-to-left carrying the running product of everything to the right.
    // The output array is reused as the prefix array, so the only extra space is `suffix`.
    int suffix = 1;
    for (int i = n - 1; i >= 0; --i) {
        out[i] *= suffix;
        suffix *= nums[i];
    }
    return out;
}

int main() {
    std::vector<int> a = {1, 2, 3, 4};
    assert((productExceptSelf(a) == std::vector<int>{24, 12, 8, 6}));

    std::vector<int> b = {-1, 1, 0, -3, 3};    // a zero: every other slot becomes 0, the zero's slot gets the rest
    assert((productExceptSelf(b) == std::vector<int>{0, 0, 9, 0, 0}));

    std::vector<int> c = {0, 0};               // two zeros: everything is 0
    assert((productExceptSelf(c) == std::vector<int>{0, 0}));
}
```

```

std::vector<int> d = {5, 7}; // minimum size per the problem (n >= 2)
assert((productExceptSelf(d) == std::vector<int>{7, 5}));

std::cout << "OK 01_product_except_self.cpp\n";
return 0;
}

```

What is new:

- `const int n = static_cast<int>(nums.size());` is Python's `n = len(nums)`, with the size turned into an `int` once (m6) so the backwards loop below is safe.
- `std::vector<int> out(n, 1);` is `out = [1] * n`: "n copies of 1".
- `for (int i = n - 1; i >= 0; --i)` is `for i in range(n - 1, -1, -1)`. It is safe *only* because `i` is an `int`; with `size_t` it would be the odometer loop from m6.
- `out[i] *= suffix;` is the same as Python. `return out;` hands the vector to the caller by value (m5); nothing is copied in practice.

Walk through the example. `nums = [1, 2, 3, 4]`, `n = 4`, `out = [1, 1, 1, 1]`.

Pass 1: `i=1: out[1] = out[0] * nums[0] = 1 * 1 = 1`. `i=2: out[2] = 1 * 2 = 2`. `i=3: out[3] = 2 * 3 = 6`. Now `out = [1, 1, 2, 6]`, the left products.

Pass 2, suffix = 1: `i=3: out[3] = 6 * 1 = 6`, suffix = 4. `i=2: out[2] = 2 * 4 = 8`, suffix = 12. `i=1: out[1] = 1 * 12 = 12`, suffix = 24. `i=0: out[0] = 1 * 24 = 24`. Result `[24, 12, 8, 6]`.

Complexity. Two passes over the list: time proportional to n , $O(n)$. Extra memory is one `int`, $O(1)$, not counting the output you were asked to produce.

Robotics. Running products and running sums are the same trick as an *integral image* or a cumulative-sum row of a costmap: precompute once in a sweep, and then any "everything except this cell" or "sum of this window" question is answered in constant time. Sensor pipelines use this to compute windowed statistics over a stream without re-reading the window each tick.

Follow-ups you may get. - *Why not multiply everything and divide by `nums[i]`?* The problem forbids division, and a zero breaks it anyway. With division allowed you would count the zeros: none, divide; exactly one, only that slot is non-zero; two or more, all zeros. - *What about overflow?* The problem promises the products fit in 32 bits. In real code use `long long` or check. - *Can you do it with only the left-to-right pass?* No: slot `i` needs numbers to its right, which a single left-to-right pass has not seen yet. Two passes are the minimum.

2. Merge Intervals (LeetCode 56, medium)

The problem in plain words. You are given a list of intervals, each a pair `[start, end]` meaning "from start to end, inclusive". Some of them overlap. Merge every group of overlapping intervals into one, and return the merged list. Intervals that only *touch* (`[1, 4]` and `[4, 5]`) count as overlapping.

```

number line:  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18
[1, 3]          [=====]
[2, 6]          [=====]
[8, 10]         [=====]
[15, 18]        [=====]

                these two overlap -> [1, 6]
                alone              -> [8, 10]
                                   [=====] -> [15, 18]

answer: [[1, 6], [8, 10], [15, 18]]

```

Edge cases: the input may be in any order; an interval may be entirely inside another (`[1, 4]` and `[2, 3]` merge to `[1, 4]`, *not* `[1, 3]`); an empty list gives an empty list; a single interval like `[5, 5]` stays as it is. Up to 10,000 intervals.

By hand. Sort the intervals by their start. Then walk them left to right keeping "the interval I am currently building". If the next one starts at or before the end of what I am building, it overlaps: stretch my end to whichever end is bigger. If it starts after my end, there is a gap: put what I built into the answer and start building from this new one.

The idea. 1. Sort by start (a lambda comparator, `s1`). 2. Put the first interval into `out`. 3. For each interval: if its start is $\leq$ the end of `out.back()`, set that end to `max(current end, this end)`. Otherwise push the interval as a new group.

The Python you would have written.

```
def merge(intervals):
    intervals.sort(key=lambda iv: iv[0])
    out = []
    for start, end in intervals:
        if out and start <= out[-1][1]:
            out[-1][1] = max(out[-1][1], end)    # overlap: stretch the last group
        else:
            out.append([start, end])             # gap: start a new group
    return out
```

In C++.

```
// 02_merge_intervals.cpp - LeetCode 56. Sort by start, then sweep and extend the last merged interval.
#include <algorithm>
#include <cassert>
#include <iostream>
#include <vector>

std::vector<std::vector<int>>> merge(std::vector<std::vector<int>>& intervals) {
    if (intervals.empty()) return {};
    // vector<int> already compares lexicographically, so this sorts by start (then end).
    // Being explicit with a lambda is what interviewers expect to see.
    std::sort(intervals.begin(), intervals.end(),
        [](const std::vector<int>& a, const std::vector<int>& b) { return a[0] < b[0]; });
    std::vector<std::vector<int>>> out;
    out.push_back(intervals[0]);
    for (const auto& iv : intervals) {
        std::vector<int>& last = out.back();    // reference: we mutate in place
        if (iv[0] <= last[1]) last[1] = std::max(last[1], iv[1]);    // overlap (touching counts): extend
        else out.push_back(iv);                // gap: start a new interval
    }
    return out;
}

int main() {
    using VV = std::vector<std::vector<int>>>;
    VV a = {{1, 3}, {2, 6}, {8, 10}, {15, 18}};
    assert((merge(a) == VV{{1, 6}, {8, 10}, {15, 18}}));

    VV b = {{1, 4}, {4, 5}};    // touching endpoints merge
    assert((merge(b) == VV{{1, 5}}));

    VV c = {{1, 4}, {2, 3}};    // fully contained: end must be max, not the later interval's end
    assert((merge(c) == VV{{1, 4}}));

    VV d = {};
    assert(merge(d).empty());

    VV e = {{5, 5}};    // single degenerate interval
    assert((merge(e) == VV{{5, 5}}));

    std::cout << "OK 02_merge_intervals.cpp\n";
    return 0;
}
```

What is new:

- `std::sort(intervals.begin(), intervals.end(), [](const std::vector<int>& a, const std::vector<int>& b) { return a[0] < b[0]; });` is `intervals.sort(key=lambda iv: iv[0])`. The lambda

answers "does a go before b?" by comparing starts (s1). Each interval here is a `std::vector<int>` of two numbers, matching LeetCode's signature.

- `if (intervals.empty()) return {};` returns an empty vector; Python did this with the `if out` check. The C++ version pushes `intervals[0]` first, so the loop's first step compares the first interval with itself, which harmlessly does nothing.
- `std::vector<int>& last = out.back();` is `out[-1]`, but as a **reference** (m1). This is the line people get wrong: without the `&`, `last` is a copy, `last[1] = ...` changes the copy, and the answer never grows.
- `for (const auto& iv : intervals)` is `for start, end in intervals`, reading each interval through a const reference (m2) so nothing is copied.
- `std::max(last[1], iv[1])` is `max(...)`. `#include <algorithm>` brings in `std::sort` and `std::max`.

Walk through the example. After sorting (already sorted here): `[[1,3],[2,6],[8,10],[15,18]]`. `out = [[1,3]]`.

- `[1,3]: 1 <= 3`, overlap; `last[1] = max(3, 3) = 3`. Nothing changes.
- `[2,6]: 2 <= 3`, overlap; `last[1] = max(3, 6) = 6`. `out = [[1,6]]`.
- `[8,10]: 8 <= 6?` No, gap; push. `out = [[1,6],[8,10]]`.
- `[15,18]: 15 <= 10?` No; push. `out = [[1,6],[8,10],[15,18]]`.

Complexity. Sorting dominates: $O(n \log n)$ time. The output takes $O(n)$ memory.

Robotics. A sensor-fusion node merges the time windows during which each sensor was valid into one "trusted" timeline. A coverage planner merges the swept segments along a row into the spans already cleaned. A 1D laser scan's free and occupied runs are merged the same way.

Follow-ups you may get. - *Why does sorting by start make one pass enough?* Once sorted, any interval that overlaps some group must overlap the group's most recent extension, so you only ever compare with `out.back()`. - *What if intervals arrive one at a time?* That is Insert Interval, the next problem. For a stream, keep a `std::map<int, int>` keyed by start and merge on insert. - *What if `[2,3]` came after `[1,4]`?* The `max` handles it: the end stays 4. Using the incoming end instead of `max` is the classic bug.

3. Insert Interval (LeetCode 57, medium)

The problem in plain words. You are given a list of intervals that is *already sorted by start and has no overlaps*, plus one new interval. Insert the new interval so the list is still sorted and still has no overlaps, merging as needed. Return the new list.

existing:	[1,2]	[3,5]	[6,7]	[8,10]		[12,16]
new:			[4-----8]			
phase 1:	[1,2] ends before 4				-> copy as is	
phase 2:	[3,5], [6,7], [8,10] touch new				-> swallow them: new grows to [3,10]	
phase 3:	[12,16] starts after 10				-> copy as is	
answer:	[1,2]	[3,10]	[12,16]			

Edge cases: the existing list may be empty (answer is just the new interval); the new interval may go before everything, after everything, or swallow everything; touching counts as overlap. Up to 10,000 intervals.

By hand. Walk the sorted list left to right. At first the intervals end before the new one even starts: copy them. Then you hit intervals that overlap the new one: do not copy them, instead widen the new interval to cover each of them. Once you reach an interval that starts after the new one's (possibly widened) end, put the new interval down and copy the rest.

The idea. One pass in three phases, with one index `i`: 1. While `intervals[i].end < new.start`: copy. 2. While `intervals[i].start <= new.end`: `new.start = min(...)`, `new.end = max(...)`, step. 3. Push new, then copy everything left.

The Python you would have written.

```
def insert(intervals, new):
    out = []
    i, n = 0, len(intervals)
    while i < n and intervals[i][1] < new[0]:           # phase 1: ends before new starts
        out.append(intervals[i]); i += 1
    while i < n and intervals[i][0] <= new[1]:          # phase 2: overlaps new
        new[0] = min(new[0], intervals[i][0])
        new[1] = max(new[1], intervals[i][1])
        i += 1
    out.append(new)
    while i < n:                                       # phase 3: the rest
        out.append(intervals[i]); i += 1
    return out
```

In C++.

```
// 03_insert_interval.cpp - LeetCode 57. Input is sorted and non-overlapping; one linear pass in three phases.
#include <algorithm>
#include <cassert>
#include <iostream>
#include <vector>

std::vector<std::vector<int>>> insert(std::vector<std::vector<int>>>& intervals, std::vector<int>& newInterval) {
    std::vector<std::vector<int>>> out;
    out.reserve(intervals.size() + 1);
    const int n = static_cast<int>(intervals.size());
    int i = 0;
    // Phase 1: everything that ends before the new interval starts is untouched.
    while (i < n && intervals[i][1] < newInterval[0]) out.push_back(intervals[i++]);
    // Phase 2: everything that overlaps gets absorbed into newInterval.
    while (i < n && intervals[i][0] <= newInterval[1]) {
        newInterval[0] = std::min(newInterval[0], intervals[i][0]);
        newInterval[1] = std::max(newInterval[1], intervals[i][1]);
        ++i;
    }
    out.push_back(newInterval);
    // Phase 3: the rest starts after the merged interval ends.
    while (i < n) out.push_back(intervals[i++]);
    return out;
}

int main() {
    using VV = std::vector<std::vector<int>>>;
    VV a = {{1, 3}, {6, 9}};
    std::vector<int> na = {2, 5};
    assert((insert(a, na) == VV{{1, 5}, {6, 9}}));

    VV b = {{1, 2}, {3, 5}, {6, 7}, {8, 10}, {12, 16}};
    std::vector<int> nb = {4, 8};
    assert((insert(b, nb) == VV{{1, 2}, {3, 10}, {12, 16}}));

    VV c = {}; // empty list: result is just the new interval
    std::vector<int> nc = {5, 7};
    assert((insert(c, nc) == VV{{5, 7}}));

    VV d = {{1, 5}}; // insert before everything
    std::vector<int> nd = {-3, 0};
    assert((insert(d, nd) == VV{{-3, 0}, {1, 5}}));

    VV e = {{1, 5}}; // insert after everything
    std::vector<int> ne = {6, 8};
    assert((insert(e, ne) == VV{{1, 5}, {6, 8}}));

    std::cout << "OK 03_insert_interval.cpp\n";
    return 0;
}
```

What is new:

- `out.reserve(intervals.size() + 1);` asks the vector for one heap shelf big enough for everything up front (m5), so `push_back` never has to move to a bigger shelf. Optional, but free.
- `out.push_back(intervals[i++]);` is `out.append(intervals[i]); i += 1` in one line. `i++` means "use the current value of `i`, then add one". It is fine here; do not use it in tricky expressions.
- `std::min` and `std::max` are Python's `min` and `max` for two values.
- `newInterval` is taken by non-const reference (`std::vector<int>&`) and changed in place, exactly like Python's `new[0] = ...` on the caller's list. That is what the given signature asks for; in your own code you would take a copy.

Walk through `[[1,3],[6,9]]` with new `[2,5]`. `i = 0`. Phase 1: does `[1,3]` end before 2? `3 < 2` is false, so phase 1 copies nothing. Phase 2: does `[1,3]` start at or before 5? `1 <= 5`, yes:

`new = [min(2,1), max(5,3)] = [1,5]`, `i = 1`. Does `[6,9]` start at or before 5? `6 <= 5`, no. Push `[1,5]`. Phase 3: copy `[6,9]`. Answer `[[1,5],[6,9]]`.

Complexity. One pass: $O(n)$ time and $O(n)$ memory for the output.

Robotics. A scheduler keeps a sorted list of "arm busy" windows and inserts a newly planned trajectory segment. A keep-out list along a path, indexed by distance along the path, gets a new obstacle zone inserted and merged with its neighbours. Anything sorted by time or arc length that must stay non-overlapping uses this exact insert.

Follow-ups you may get. - *Can you find the insertion point faster than scanning?* Use `std::lower_bound` (binary search) to find the first overlap in $O(\log n)$, but shifting the vector to insert is $O(n)$ anyway. A `std::map<int,int>` keyed by start gives $O(\log n + k)$ where k intervals are merged. - *Do it in place?* Erase the overlapped range and insert the merged interval: `intervals.erase(first, last); intervals.insert(first, merged);`. - *Why `<` in phase 1 but `<=` in phase 2?* Because touching counts as overlap: `[1,2]` and a new `[2,5]` must merge, so "ends strictly before" is the condition for leaving an interval alone.

4. 3Sum (LeetCode 15, medium)

The problem in plain words. Given a list of integers, find every *unique* group of three numbers (from three different positions) that add up to zero. Return the groups in any order; two groups with the same three values count as the same group and must appear only once.

```
nums = [-1, 0, 1, 2, -1, -4]

sorted:  -4  -1  -1  0  1  2
index:    0  1  2  3  4  5

-1 + -1 + 2 = 0    -> [-1, -1, 2]
-1 + 0 + 1 = 0    -> [-1, 0, 1]
(the second -1 gives the same groups again, so it must be skipped)

answer: [[-1, -1, 2], [-1, 0, 1]]
```

Edge cases: fewer than three numbers gives an empty answer; `[0, 0, 0, 0]` gives exactly one group `[0, 0, 0]`; no group at all (`[0, 1, 1]`) gives an empty list. Up to 3,000 numbers.

By hand. Sort the numbers first. Pick the smallest as the "anchor" and now you need two more numbers that add to minus the anchor. With the rest sorted, put one finger on the left end and one on the right end. If the three add to less than zero, the sum is too small, so move the left finger right (to a bigger number). If more than zero, move the right finger left. If exactly zero, write it down, then move both fingers past any repeats. When the fingers meet, move the anchor to the next *different* number and repeat.

The idea. 1. Sort. For each anchor `i`: if `nums[i] > 0`, stop (three positives cannot sum to zero). If `nums[i] == nums[i-1]`, skip (same anchor, same groups). 2. `lo = i + 1`, `hi = n - 1`. While `lo < hi`: sum too small, `++lo`; too big, `--hi`; zero, record it and step both fingers past duplicates.

The Python you would have written.

```
def threeSum(nums):
    nums.sort()
    n = len(nums)
    out = []
    for i in range(n - 2):
        if nums[i] > 0:
            break
        if i > 0 and nums[i] == nums[i - 1]:
            continue
        lo, hi = i + 1, n - 1
        while lo < hi:
            s = nums[i] + nums[lo] + nums[hi]
            if s < 0:
                lo += 1
            elif s > 0:
                hi -= 1
            else:
                out.append([nums[i], nums[lo], nums[hi]])
                while lo < hi and nums[lo] == nums[lo + 1]: lo += 1
                while lo < hi and nums[hi] == nums[hi - 1]: hi -= 1
                lo += 1; hi -= 1
    return out
```

In C++.

```
// 04_three_sum.cpp - LeetCode 15. Sort, fix one element, two pointers for the other two, skip duplicates.
#include <algorithm>
#include <cassert>
#include <iostream>
#include <vector>

std::vector<std::vector<int>>> threeSum(std::vector<int>& nums) {
    std::vector<std::vector<int>>> out;
    std::sort(nums.begin(), nums.end());
    const int n = static_cast<int>(nums.size());
    for (int i = 0; i < n - 2; ++i) {
        if (nums[i] > 0) break;
        if (i > 0 && nums[i] == nums[i - 1]) continue;
        int lo = i + 1, hi = n - 1;
        while (lo < hi) {
            const int sum = nums[i] + nums[lo] + nums[hi];
            if (sum < 0) ++lo;
            else if (sum > 0) --hi;
            else {
                out.push_back({nums[i], nums[lo], nums[hi]});
                // Skip duplicates on both sides before moving on, otherwise the same triple repeats.
                while (lo < hi && nums[lo] == nums[lo + 1]) ++lo;
                while (lo < hi && nums[hi] == nums[hi - 1]) --hi;
                ++lo; --hi;
            }
        }
    }
    return out;
}

int main() {
    using VV = std::vector<std::vector<int>>>;
    auto sorted = []<VV v> { std::sort(v.begin(), v.end()); return v; }; // answer order is unspecified

    std::vector<int> a = {-1, 0, 1, 2, -1, -4};
    assert(sorted(threeSum(a)) == sorted(VV{{-1, -1, 2}, {-1, 0, 1}}));

    std::vector<int> b = {0, 1, 1};
    assert(threeSum(b).empty());

    std::vector<int> c = {0, 0, 0, 0};
    assert((threeSum(c) == VV{{0, 0, 0}}));
```

```

std::vector<int> d = {-2, 0, 0, 2, 2};           // duplicate skipping on lo and hi
assert((threeSum(d) == VV{{-2, 0, 2}}));

std::vector<int> e = {1, 2};                   // fewer than three elements
assert(threeSum(e).empty());

std::cout << "OK 04_three_sum.cpp\n";
return 0;
}

```

What is new:

- `std::sort(nums.begin(), nums.end());` is `nums.sort()`. No comparator: ints sort ascending by themselves (s1).
- `for (int i = 0; i < n - 2; ++i)` is `for i in range(n - 2)`. With `n` an `int` (m6), a list of two numbers gives `n - 2 == 0` and the loop never runs, which is correct. With `size_t`, `n - 2` on a list of one would roll round to a huge number.
- `break` and `continue` mean the same as in Python.
- `int lo = i + 1, hi = n - 1;` declares two ints on one line, Python's `lo, hi = i + 1, n - 1`.
- `out.push_back({nums[i], nums[lo], nums[hi]});` is `out.append(...)`; the braces build the three-element vector on the spot.
- `++lo; --hi;` is `lo += 1; hi -= 1`.

Walk through the example. Sorted: `[-4, -1, -1, 0, 1, 2]`, `n = 6`.

- `i=0` (anchor -4): `lo=1, hi=5: -4-1+2 = -3 < 0, lo=2. -4-1+2 = -3, lo=3. -4+0+2 = -2, lo=4. -4+1+2 = -1, lo=5`. Fingers meet. Nothing found.
- `i=1` (anchor -1): `lo=2, hi=5: -1-1+2 = 0`. Record `[-1, -1, 2]`. Skip repeats: `nums[2]` vs `nums[3]` differ, `nums[5]` vs `nums[4]` differ. `lo=3, hi=4: -1+0+1 = 0`. Record `[-1, 0, 1]`. `lo=4, hi=3`, stop.
- `i=2` (anchor -1 again): `nums[2] == nums[1]`, skip. This is what stops the duplicate groups.
- `i=3` (anchor 0): `lo=4, hi=5: 0+1+2 = 3 > 0, hi=4`. Stop.
- `i=4` is not `< n-2 = 4`; loop ends. Answer `[[-1, -1, 2], [-1, 0, 1]]`.

Complexity. The sort is $O(n \log n)$; then for each of n anchors the two fingers together walk at most n steps: $O(n^2)$ time. Extra memory is $O(1)$ beyond the sort and the output.

Robotics. Two fingers walking inward over sorted data is how you match events by timestamp: pairing each lidar scan with the nearest IMU sample, or finding pairs of measurements whose time difference is within a tolerance. The discipline of skipping repeats is the same one you need when several detections fall into the same grid cell and must be counted once.

Follow-ups you may get. - Why compare with `nums[i-1]` and not `nums[i+1]` when skipping anchors?

Skipping ahead would throw away the *first* -1 and lose `[-1, -1, 2]`. You skip an anchor only if the previous anchor was the same value and has already done the work. - *3Sum Closest, or 4Sum?* Same skeleton. For closest, track the best `|sum - target|` instead of testing for zero. For 4Sum, add one more outer loop: $O(n^3)$. - *Is it OK that the sort changes the input?* The signature takes a non-const reference, so the problem allows it. If the caller needs the original order, copy first.

5. Container With Most Water (LeetCode 11, medium)

The problem in plain words. You are given a list of heights. Each height is a vertical wall standing at position `i` on the ground. Pick two walls; together with the ground they form a container. It holds water up to the height of the *shorter* wall, across the distance between them. Return the largest amount of water any pair of walls can hold, that is the largest `min(height[i], height[j]) * (j - i)`.

```
height = [1, 8, 6, 2, 5, 4, 8, 3, 7]
```

```
8 |      #                #
```

```

7 |   #           #   #
6 |   # #       #   #
5 |   # #   #   #   #
4 |   # #   # # #   #
3 |   # #   # # # # #
2 |   # # # # # # # #
1 | # # # # # # # # #
  +-----+
    0  1  2  3  4  5  6  7  8

```

best pair: wall 1 (height 8) and wall 8 (height 7)

water = $\min(8, 7) * (8 - 1) = 7 * 7 = 49$

Edge cases: two walls of height 1 hold 1; a single wall holds nothing (0); the outer two walls can be the answer ([4, 3, 2, 1, 4] gives 16). Up to 100,000 walls, so trying every pair is too slow.

By hand. Start with the two outermost walls: that is the widest container you can make. Now ask: which wall is worth moving? The shorter one. Moving the taller wall inward can never help, because the water level is set by the shorter wall and the width only shrinks. So move the shorter wall one step inward, measure again, and keep going until the two walls meet. Remember the biggest measurement.

The idea. 1. $lo = 0$, $hi = n - 1$, $best = 0$. 2. Measure $\min(\text{height}[lo], \text{height}[hi]) * (hi - lo)$; keep the max. 3. If $\text{height}[lo] < \text{height}[hi]$ move lo right, else move hi left. Stop when they meet.

The Python you would have written.

```

def maxArea(height):
    lo, hi = 0, len(height) - 1
    best = 0
    while lo < hi:
        best = max(best, min(height[lo], height[hi]) * (hi - lo))
        if height[lo] < height[hi]:
            lo += 1          # the shorter wall is on the left: move it
        else:
            hi -= 1
    return best

```

In C++.

```

// 05_container_water.cpp - LeetCode 11. Two pointers from the outside in; always move the shorter wall.
#include <algorithm>
#include <cassert>
#include <iostream>
#include <vector>

int maxArea(std::vector<int>& height) {
    int lo = 0, hi = static_cast<int>(height.size()) - 1;
    int best = 0;
    while (lo < hi) {
        best = std::max(best, std::min(height[lo], height[hi]) * (hi - lo));
        // The shorter wall bounds the area. Moving the taller one can only shrink width without
        // raising the limit, so the shorter side is the only move that might improve.
        if (height[lo] < height[hi]) ++lo;
        else --hi;
    }
    return best;
}

int main() {
    std::vector<int> a = {1, 8, 6, 2, 5, 4, 8, 3, 7};
    assert(maxArea(a) == 49);

    std::vector<int> b = {1, 1};
    assert(maxArea(b) == 1);

    std::vector<int> c = {4, 3, 2, 1, 4};    // outer walls are the answer
    assert(maxArea(c) == 16);
}

```

```

std::vector<int> d = {1, 2, 1};
assert(maxArea(d) == 2);

std::vector<int> e = {7};           // single wall: no container
assert(maxArea(e) == 0);

std::cout << "OK 05_container_water.cpp\n";
return 0;
}

```

What is new:

- `int lo = 0, hi = static_cast<int>(height.size()) - 1;` is `lo, hi = 0, len(height) - 1`. The cast matters (m6): on an empty list `size() - 1` would roll round, but `static_cast<int>(0) - 1` is `-1`, and while `(lo < hi)` simply does not run.
- `std::max(best, std::min(height[lo], height[hi]) * (hi - lo))` is the same line as Python with `std::` in front. Both come from `<algorithm>`.
- `++lo` and `--hi` are `lo += 1` and `hi -= 1`.

Walk through the example. `n = 9, lo = 0, hi = 8, best = 0`.

lo	hi	heights	water	best	move
0	8	1, 7	$1 \times 8 = 8$	8	$1 < 7$: <code>lo → 1</code>
1	8	8, 7	$7 \times 7 = 49$	49	$8 < 7$ false: <code>hi → 7</code>
1	7	8, 3	$3 \times 6 = 18$	49	<code>hi → 6</code>
1	6	8, 8	$8 \times 5 = 40$	49	equal, else branch: <code>hi → 5</code>
1	5	8, 4	$4 \times 4 = 16$	49	<code>hi → 4</code>
1	4	8, 5	$5 \times 3 = 15$	49	<code>hi → 3</code>
1	3	8, 2	$2 \times 2 = 4$	49	<code>hi → 2</code>
1	2	8, 6	$6 \times 1 = 6$	49	<code>hi → 1</code>

`lo == hi`, stop. Answer 49.

Complexity. The two fingers together take at most n steps: $O(n)$ time, $O(1)$ memory.

Robotics. "Shrink from the outside and only ever move the limiting side" is the same reasoning you use when tightening a time window between two sensors' valid ranges, or when picking the widest clearance corridor between two obstacle profiles along a scan line: the more constraining side is the only one worth moving.

Follow-ups you may get. - *Prove that moving the taller wall never helps.* Say `height[lo] ≤ height[hi]`. Any container using `lo` and some `j` between them has water at most `height[lo] * (j - lo)`, which is less than `height[lo] * (hi - lo)`, the one we just measured. So `lo` can never do better; retire it. - *What if the two walls are equal?* Move either; the code moves `hi`. Both choices are safe by the argument above applied to each side. - *Trapping Rain Water (LeetCode 42)?* Also two fingers, but you sum water over every position using running left and right maxima. Chapter 05.

6. Rotate Image (LeetCode 48, medium)

The problem in plain words. You are given a square grid of numbers, n rows by n columns. Rotate it 90 degrees clockwise, *in place*: change the grid you were given rather than building a new one.

before	after (turned a quarter turn to the right)
1 2 3	7 4 1

```
4 5 6    -->    8 5 2
7 8 9          9 6 3
```

The top row 1 2 3 became the right column, read top to bottom. Edge cases: a 1x1 grid does not change; a 2x2 `[[1,2],[3,4]]` becomes `[[3,1],[4,2]]`; four rotations bring the original back. `n` is at most 20.

By hand. Rotating by hand means moving every number to a new place at once, which is hard to do in place. But there is a two-step trick that is easy. First *transpose*: flip the grid over its main diagonal (the line from top-left to bottom-right), so rows become columns. Then *reverse each row*. Do those two easy things and the grid has been rotated.

original		step 1: transpose (flip over the \ diagonal)		step 2: reverse each row
1 2 3		1 4 7		7 4 1
4 5 6	---->	2 5 8	---->	8 5 2
7 8 9		3 6 9		9 6 3

Look at step 1: the number at row `i`, column `j` moved to row `j`, column `i`. The diagonal 1 5 9 did not move. Look at step 2: each row was simply read backwards.

The idea. 1. For every pair of positions above the diagonal (`j > i`), swap `matrix[i][j]` with `matrix[j][i]`. Starting `j` at `i + 1` swaps each pair exactly once. 2. Reverse every row.

The Python you would have written.

```
def rotate(matrix):
    n = len(matrix)
    for i in range(n):
        for j in range(i + 1, n):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j] # transpose
    for row in matrix:
        row.reverse() # flip each row
```

In C++.

```
// 06_rotate_image.cpp - LeetCode 48. Rotate an n x n matrix 90 degrees clockwise in place:
// transpose, then reverse each row.
#include <algorithm>
#include <cassert>
#include <iostream>
#include <utility>
#include <vector>

void rotate(std::vector<std::vector<int>>& matrix) {
    const int n = static_cast<int>(matrix.size());
    // Transpose: swap across the main diagonal. j starts at i+1 so each pair is swapped once.
    for (int i = 0; i < n; ++i)
        for (int j = i + 1; j < n; ++j)
            std::swap(matrix[i][j], matrix[j][i]);
    // Reverse each row: transpose + horizontal flip == 90 degrees clockwise.
    for (auto& row : matrix) std::reverse(row.begin(), row.end());
}

int main() {
    using VV = std::vector<std::vector<int>>;
    VV a = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    rotate(a);
    assert((a == VV{{7, 4, 1}, {8, 5, 2}, {9, 6, 3}}));

    VV b = {{5, 1, 9, 11}, {2, 4, 8, 10}, {13, 3, 6, 7}, {15, 14, 12, 16}};
    rotate(b);
    assert((b == VV{{15, 13, 2, 5}, {14, 3, 4, 1}, {12, 6, 8, 9}, {16, 7, 10, 11}}));

    VV c = {{1}}; // 1x1 is a fixed point
```



```

rotate(c);
assert((c == VV{{1}}));

VV d = {{1, 2}, {3, 4}};
rotate(d);
assert((d == VV{{3, 1}, {4, 2}}));
rotate(d); rotate(d); rotate(d);           // four rotations = identity
assert((d == VV{{1, 2}, {3, 4}}));

std::cout << "OK 06_rotate_image.cpp\n";
return 0;
}

```

What is new:

- `void rotate(std::vector<std::vector<int>>& matrix)` returns nothing and takes the grid by reference (m1), so the caller's grid is the one that changes. Without the `&` you would rotate a copy and the caller would see nothing.
- `std::swap(matrix[i][j], matrix[j][i]);` is Python's `a, b = b, a`. It comes from `<utility>`.
- `for (auto& row : matrix) std::reverse(row.begin(), row.end());` is for `row in matrix: row.reverse()`. The `&` on `auto& row` is essential: `auto row` would be a copy of the row, and reversing the copy leaves the grid untouched.
- `std::reverse(first, last)` reverses whatever lies between two bookmarks (s4); here, the whole row.

Walk through the example. `n = 3`.

Transpose: `i=0`: swap `(0,1)` with `(1,0)`: 2 and 4 trade places. Swap `(0,2)` with `(2,0)`: 3 and 7. `i=1`: swap `(1,2)` with `(2,1)`: 6 and 8. `i=2`: `j` would start at 3, nothing to do. Grid is now `[[1,4,7],[2,5,8],[3,6,9]]`.

Reverse rows: `[1,4,7] → [7,4,1]`; `[2,5,8] → [8,5,2]`; `[3,6,9] → [9,6,3]`. Done.

Complexity. Every cell is touched a constant number of times: $O(n^2)$ time (there are n^2 cells, so this is one pass). $O(1)$ extra memory.

Robotics. Rotating an occupancy grid by 90 degrees when the map convention changes (image (`row`, `col`) with `row` growing downward versus map (`x`, `y`) with `y` growing upward) is exactly this operation: the transpose is the axis swap and the row reversal is the axis flip. It also explains, in miniature, why the inverse of a rotation matrix is its transpose.

Follow-ups you may get. - *Counter-clockwise?* Transpose, then reverse each *column* (or reverse the rows first, then transpose). - *Not square?* An `m`-by-`n` grid cannot be rotated in place inside the same buffer. Build an `n`-by-`m` output with `out[j][n - 1 - i] = in[i][j]`. - *What goes wrong if `j` starts at 0 instead of `i + 1`?* Every pair is swapped twice and the grid ends up exactly as it started.

7. Top K Frequent Elements (LeetCode 347, medium)

The problem in plain words. Given a list of integers and a number `k`, return the `k` values that appear most often. The problem promises the answer is unique, and you may return the values in any order.

```

nums = [1, 1, 1, 2, 2, 3], k = 2

count:   1 appears 3 times
         2 appears 2 times
         3 appears 1 time

the 2 most frequent: 1 and 2      answer: [1, 2] (or [2, 1])

```

Edge cases: a single element with `k = 1` returns it; `k` may equal the number of distinct values (return them all); negative numbers are fine. Up to 100,000 numbers, and `k` is never larger than the number of distinct values.

By hand. First make a tally: one line per distinct value with how many times it appears. Then you need the k biggest tallies. Sorting the whole tally works, but there is a neater way that never keeps more than k things in hand: keep a small pile of "candidates so far" that always shows the *weakest* candidate on top. Add each value's tally to the pile; if the pile now has more than k things, throw away the weakest. At the end the pile holds exactly the k strongest.

The idea. 1. Count with an `unordered_map` (s4): `++freq[x]`. 2. For each `(count, value)` pair, push it onto a min-heap (s2, smallest count on top). If the heap size passes k, pop the top (the weakest). 3. Empty the heap into the answer.

The Python you would have written.

```
from collections import Counter
import heapq

def topKFrequent(nums, k):
    freq = Counter(nums)
    heap = []
    for val, cnt in freq.items():
        heapq.heappush(heap, (cnt, val))    # heapq is a min-heap: weakest on top
        if len(heap) > k:
            heapq.heappop(heap)            # throw away the weakest
    return [val for cnt, val in heap]
```

In C++.

```
// 07_top_k_frequent.cpp - LeetCode 347. Count with a hash map, keep the k largest counts in a min-heap.
#include <algorithm>
#include <cassert>
#include <functional>
#include <iostream>
#include <queue>
#include <unordered_map>
#include <utility>
#include <vector>

std::vector<int> topKFrequent(std::vector<int>& nums, int k) {
    std::unordered_map<int, int> freq;
    for (int x : nums) ++freq[x];

    // Min-heap on (count, value): the top is the weakest of the current k candidates.
    // When the heap exceeds k, pop the weakest. Heap size never exceeds k+1 -> O(n log k).
    using CountVal = std::pair<int, int>;
    std::priority_queue<CountVal, std::vector<CountVal>, std::greater<CountVal>> heap;
    for (const auto& [val, cnt] : freq) {
        heap.push({cnt, val});
        if (static_cast<int>(heap.size()) > k) heap.pop();
    }
    std::vector<int> out;
    out.reserve(static_cast<std::size_t>(k));
    while (!heap.empty()) {
        out.push_back(heap.top().second);
        heap.pop();
    }
    return out;    // any order is accepted by the problem
}

int main() {
    auto sorted = [](std::vector<int> v) { std::sort(v.begin(), v.end()); return v; };

    std::vector<int> a = {1, 1, 1, 2, 2, 3};
    assert(sorted(topKFrequent(a, 2)) == (std::vector<int>{1, 2}));

    std::vector<int> b = {1};
    assert(sorted(topKFrequent(b, 1)) == (std::vector<int>{1}));

    std::vector<int> c = {4, 4, 4, 5, 5, 6, 6, 7};    // k equal to the number of distinct values
    assert(sorted(topKFrequent(c, 4)) == (std::vector<int>{4, 5, 6, 7}));
}
```

```

std::vector<int> d = {-1, -1, 2, 2, 2, 3};           // negatives are fine as hash keys
assert(sorted(topKFrequent(d, 1)) == (std::vector<int>{2}));

std::cout << "OK 07_top_k_frequent.cpp\n";
return 0;
}

```

What is new:

- `std::unordered_map<int, int> freq;` for `(int x : nums) ++freq[x];` is `Counter(nums)`. `freq[x]` creates the entry at 0 the first time (s4), then `++` bumps it.
- `using CountVal = std::pair<int, int>;` names the pair type (count, value) so the next line is readable.
- `std::priority_queue<CountVal, std::vector<CountVal>, std::greater<CountVal>> heap;` is the min-heap of pairs from s2: the pair with the smallest first (the count) sits on top. This is why the pair is (count, value) and not (value, count).
- `for (const auto& [val, cnt] : freq)` is for `val, cnt` in `freq.items()`, with the pair unpacked in place.
- `if (static_cast<int>(heap.size()) > k) heap.pop();` is `if len(heap) > k`. The cast keeps an unsigned `size()` from being compared with a signed `k` (m6).
- `heap.top().second` reads the value half of the top pair; then `heap.pop()` removes it (s2: look, then pop).

Walk through the example. `freq = {1: 3, 2: 2, 3: 1}`, `k = 2`. (An `unordered_map` may hand these out in any order; the end result is the same. Take this order.)

<code>push (3, 1)</code>	<code>heap: top -> (3,1)</code>	<code>size 1</code>
<code>push (2, 2)</code>	<code>heap: top -> (2,2) (3,1)</code>	<code>size 2</code>
<code>push (1, 3)</code>	<code>heap: top -> (1,3) (2,2) (3,1)</code>	<code>size 3 > 2: pop the top, (1,3)</code>
	<code>heap: top -> (2,2) (3,1)</code>	<code>size 2</code>
<code>drain:</code>	<code>out = [2, 1]</code>	

Any order is accepted, so `[2, 1]` is a correct answer.

Complexity. Counting is one pass, $O(n)$. Each of the d distinct values costs a push and maybe a pop on a heap that never holds more than $k+1$ items: $O(d \log k)$. Total $O(n \log k)$ time, $O(n)$ memory for the tally.

Robotics. "The k object classes seen most often in the last minute", "the k map cells hit most by a lidar sweep", "the k loop-closure candidates with the most votes in a place recogniser": count into a hash map, keep a bounded heap. The same bounded min-heap is how a KD-tree search keeps the k nearest neighbours found so far.

Follow-ups you may get. - *Why a min-heap and not a max-heap?* With a max-heap you would pop k times from a heap of all d values: $O(d \log d)$. The size- k min-heap shows you the weakest candidate, so evicting is $O(\log k)$. - *Can you do it in $O(n)$?* Bucket sort by count: `buckets[count].push_back(value)` with $n + 1$ buckets, then walk from the highest bucket down until you have k values. - *What about ties?* The problem promises there are none. If there could be, put the tie-break in the pair, for example `(count, -value)`.

8. Min Stack (LeetCode 155, medium)

The problem in plain words. Design a stack (the pile of plates from s3) with the usual `push(x)`, `pop()`, and `top()`, plus one extra operation `getMin()` that returns the smallest value currently anywhere in the stack. Every one of the four operations must take constant time, no matter how many plates are on the pile. `pop`, `top` and `getMin` are only ever called on a non-empty stack.

<code>push(-2)</code>	<code>push(0)</code>	<code>push(-3)</code>	<code>getMin() -> -3</code>
<code>pop()</code>			<code>top() -> 0</code>
			<code>getMin() -> -2</code>

Edge cases: duplicates of the minimum (`push(1)`, `push(1)`, `push(2)`, `pop()`, `pop()` must still report `getMin() == 1`); a rising sequence where the minimum never changes. Up to 30,000 operations.

By hand. Keeping "the minimum so far" as a single number breaks the moment you pop it: what was the minimum *before* it? You would have to search. So keep a *second* pile that remembers the history of minimums. Whenever you push a value that is at least as small as the current minimum, also push it onto the second pile. Whenever you pop a value that equals the top of the second pile, pop that too. The top of the second pile is always the answer to `getMin`.

Here is the state of both piles after each operation in the example (tops on the right):

operation	data_	mins_	getMin()	why
push(-2)	[-2]	[-2]	-2	mins_ was empty, so push
push(0)	[-2, 0]	[-2]	-2	0 <= -2 is false: not pushed on mins_
push(-3)	[-2, 0, -3]	[-2, -3]	-3	-3 <= -2: pushed on both
pop()	[-2, 0]	[-2]	-2	popped -3 equals mins_ top, so pop it there too
top()	-> 0			

The idea. - `push(v)`: push on `data_`; if `mins_` is empty or `v <= mins_.top()`, push on `mins_` too. - `pop()`: if `data_.top() == mins_.top()`, pop `mins_`; then pop `data_`. - `top()`: `data_.top()`. `getMin()`: `mins_.top()`.

The `<=` (not `<`) is what makes duplicates work: pushing 1, 1 puts *two* 1s on `mins_`, so popping one of them leaves the other as the minimum.

The Python you would have written.

```
class MinStack:
    def __init__(self):
        self.data = []
        self.mins = []

    def push(self, val):
        self.data.append(val)
        if not self.mins or val <= self.mins[-1]:
            self.mins.append(val)

    def pop(self):
        if self.data[-1] == self.mins[-1]:
            self.mins.pop()
        self.data.pop()

    def top(self):
        return self.data[-1]

    def getMin(self):
        return self.mins[-1]
```

In C++.

```
// 08_min_stack.cpp - LeetCode 155. A stack with O(1) getMin: a second stack tracks the running minimum.
#include <cassert>
#include <iostream>
#include <stack>

class MinStack {
public:
    MinStack() = default;

    void push(int val) {
        data_.push(val);
        // mins_ holds the minimum of everything at or below this position.
        // Push on <= (not <) so duplicates of the minimum survive a pop of one copy.
        if (mins_.empty() || val <= mins_.top()) mins_.push(val);
    }

    void pop() {
        if (data_.top() == mins_.top()) mins_.pop(); // this element was the current minimum
        data_.pop();
    }

    int top() const { return data_.top(); }
```

```

int getMin() const { return mins_.top(); }

private:
    std::stack<int> data_;
    std::stack<int> mins_;
};

int main() {
    MinStack s;
    s.push(-2); s.push(0); s.push(-3);
    assert(s.getMin() == -3);
    s.pop();
    assert(s.top() == 0);
    assert(s.getMin() == -2);

    MinStack dup;                // duplicate minimums: popping one must not lose the other
    dup.push(1); dup.push(1); dup.push(2);
    assert(dup.getMin() == 1);
    dup.pop(); dup.pop();
    assert(dup.getMin() == 1);
    dup.pop();

    MinStack rising;             // increasing pushes: mins_ stays size 1
    rising.push(5); rising.push(6); rising.push(7);
    assert(rising.getMin() == 5 && rising.top() == 7);
    rising.pop(); rising.pop(); rising.pop();
    rising.push(3);
    assert(rising.getMin() == 3);

    std::cout << "OK 08_min_stack.cpp\n";
    return 0;
}

```

What is new:

- `class MinStack { public: ... private: ... };` is the box-with-buttons from m3. The two piles `data_` and `mins_` are private compartments (the trailing underscore is a common naming habit for members); the four operations are public buttons. Python's `self.data` becomes `data_`.
- `MinStack()` = default; says "the constructor does nothing special": the two `std::stack`s build themselves empty, like `self.data = []`. There is no destructor because both compartments clean up after themselves: rule of zero (m3).
- `std::stack<int> data_;` is the pile of plates from s3 (push, pop, top, empty).
- `int top() const` and `int getMin() const`: these only read, so they carry the `const` sticker (m2) and can be called on a `const MinStack&`.
- `void push(int val)` and `void pop()` change the compartments, so they are not `const`.

Walk through the example. That is exactly the table above: after `push(-2)`, `push(0)`, `push(-3)` we have `data_ = [-2, 0, -3]` and `mins_ = [-2, -3]`, so `getMin()` is -3. `pop()` sees `data_.top() == -3 == mins_.top()`, pops both: `data_ = [-2, 0]`, `mins_ = [-2]`. `top()` is 0 and `getMin()` is -2.

For the duplicate case: `push(1)`, `push(1)`, `push(2)` gives `data_ = [1, 1, 2]` and, thanks to `<=`, `mins_ = [1, 1]`. `pop()` removes the 2 (not equal to 1, so `mins_` untouched). `pop()` removes a 1 and one 1 from `mins_`. `mins_ = [1]`, so `getMin()` is still 1.

Complexity. Each operation touches the tops of two piles: $O(1)$ time. Memory is $O(n)$ in the worst case (a falling sequence puts every value on both piles).

Robotics. A pile of running aggregates is the shape of any scoped state with cheap undo: pushing a transform onto a chain while keeping the cumulative pose, or pushing a cost onto a planner's backtracking stack while tracking the best cost seen so far. The two-stack trick works for any "combine" operation that you can recompute from the history: min, max, sum, matrix product.

Follow-ups you may get. - *One stack instead of two?* Store pairs (value, minSoFar) on a single stack. Same time, about twice the memory per element. - *What breaks with < instead of <=?* Push 1, 1, pop once: the only 1 on mins_ is gone and getMin() reports the wrong thing (or reads an empty stack). - *O(1) extra memory?* There is a trick that stores encoded differences $2*v - min$ in a long long stack. Know that it exists; it is rarely expected.

Part C — Check yourself

Answer each in plain words, with a drawing where one helps. If you cannot, reread that section.

1. Draw `int a = 5; int* p = &a; int& r = a;` as boxes. Then show what `*p = 6` and `r = 7` do.
2. What is `x` after `void f(int a) { ++a; }` is called as `f(x)` with `x = 1`? Which two signatures would make the caller see the change?
3. Name three things a reference cannot do that a pointer can.
4. Read `const int* p` and `int* const p` out in words, and say which assignment each forbids.
5. Why do we pass a `std::string` as `const std::string&` instead of `std::string`?
6. A getter is written without `const`. What stops compiling, and what is the right fix?
7. Two objects `a` then `b` are created in one block. In what order do their destructors run, and who calls them?
8. What does RAII mean in the library-book picture, and name two standard classes that do it.
9. State the rule of zero in one sentence.
10. Draw the `shared_ptr` counter for: create `s1`, copy to `s2` inside a block, leave the block, then `s1.reset()`. When is the object deleted? What does a `weak_ptr` see at each step?
11. Why is `int& bad() { int x = 1; return x; }` wrong? Draw the stack trays before and after the return.
12. Why does `while (--n >= 0)` never stop when `n` is a `size_t`? Write two safe backwards loops.
13. What question does a `std::sort` comparator answer, and why must it use `<` rather than `<=`?
14. Write the type of a priority queue of (double, int) pairs with the smallest double on top.
15. What does `m["z"]` do on a missing key, what should you use for a read-only check, and what is wrong with `m.erase(it); ++it;?`